

Facebook Weekly Rank Diffusion: Master EDA + Model Fit

Contents

Setup	1
Load weekly Facebook data and construct core panel	4
Data QA and coverage	5
Stylized fact A: Capital Distribution Curve (CDC) and stability	7
CDC stability over time windows	8
Endpoint-level forward growth distributions (v2/v3)	10
Stylized fact B: Rank-dependent drift and volatility (rank-slot increments)	12
Smoothed per-rank moments (v9+)	13
Stylized fact C: Durable change targets (v7–v10)	15
Martingale-style diagnostic (v7)	17
Stylized fact D: Gap structure via $\text{Xi_k}(t) = \log(w_k / w_{\{k+1\}})$ (v11)	18
v11-style $\sigma_hat(k)$ construction	20
Stability across time (quarterly summary)	21
Bootstrap CI over time (v11-style)	22
Stylized fact C (expanded): Turnover and rank mobility (new)	24
9.1 Top-K membership turnover (set overlap)	24
9.2 Rank-conditioned mobility: where does the endpoint at rank k go?	25
9.3 Rank transition matrix (deciles + out-of-topK)	28
Stylized fact E: Common shocks vs idiosyncratic movement (new)	29
Heavy-tail / jump diagnostics for increments (new)	31
Simulation framework (v8–v11) + unified fit metrics	32
12.1 Empirical entrant pool for entry/exit	32
12.2 Simulator (Gaussian increments with smoothed moments; optional Xi output)	33
12.3 Fit metrics (CDC, durable change, Xi)	35
Baseline simulation run + comparison (CDC, durable, Xi)	36
Micro-level movement validation (v12) + rank-slot comparator	37
14.1 Empirical micro object (identity-based, v12)	37
14.2 Simulation micro object (rank-slot, v12 style)	38
14.3 Empirical rank-slot micro comparator (new)	39
Micro fit across ranks (v12-style)	41
Sigma calibration loop (v10) (optional but included)	44
Sigma calibration by horizon (v10-style) (optional)	46
Grid search over (μ , σ , h) (v9-style) (optional)	47
Alternative model: nonparametric bootstrap increments (v2/v6) (optional)	48
Notes for SPT-facing interpretation (non-quantitative)	51

Setup

```
library(tidyverse)
library(arrow)
library(lubridate)
library(scales)
library(here)
library(broom)
```

```

fb_weekly_parquet_path <- params$fb_weekly_parquet
if (!file.exists(fb_weekly_parquet_path)) {
  candidates <- c(
    here::here("fb_ranked_weekly.parquet"),
    here::here("Facebook", "fb_ranked_weekly.parquet")
  )
  fb_weekly_parquet_path <- candidates[file.exists(candidates)][1]
}

if (is.na(fb_weekly_parquet_path) || !file.exists(fb_weekly_parquet_path)) {
  stop(
    "Weekly parquet not found. Tried: ",
    paste(c(params$fb_weekly_parquet,
      here::here("fb_ranked_weekly.parquet"),
      here::here("Facebook", "fb_ranked_weekly.parquet")),
      collapse = ", ")
  )
}

```

```

set.seed(params$seed)

```

```

coerce_numeric <- function(x) {
  if (is.null(x)) return(NA_real_)
  if (is.numeric(x)) return(as.numeric(x))
  if (is.character(x)) {
    return(suppressWarnings(readr::parse_number(x)))
  }
  suppressWarnings(as.numeric(x))
}

```

```

coerce_numeric_vec <- function(x) {
  if (is.null(x)) return(numeric(0))
  if (is.list(x)) x <- unlist(x, use.names = FALSE)
  if (is.numeric(x)) return(as.numeric(x))
  if (is.character(x)) return(readr::parse_number(x))
  suppressWarnings(as.numeric(x))
}

```

```

K_cut_target <- coerce_numeric(params$K_cut_target)
K_tail_buffer <- coerce_numeric(params$K_tail_buffer)
Smoothing_h <- coerce_numeric(params$smoothing_h)
bootstrap_B <- coerce_numeric(params$bootstrap_B)
bootstrap_k_min <- coerce_numeric(params$bootstrap_k_min)
sim_T_weeks <- as.integer(coerce_numeric(params$sim_T_weeks))
sim_n_paths <- as.integer(coerce_numeric(params$sim_n_paths))
sim_mu <- coerce_numeric(params$sim_mu)
sim_sigma <- coerce_numeric(params$sim_sigma)
sim_entry_frac <- coerce_numeric(params$sim_entry_frac)
sim_K_xi <- as.integer(coerce_numeric(params$sim_K_xi))
pca_K <- as.integer(coerce_numeric(params$pca_K))

```

```

horizons_growth <- as.integer(coerce_numeric_vec(params$horizons_growth))
horizons_durable <- as.integer(coerce_numeric_vec(params$horizons_durable))

```

```

horizons_micro <- as.integer(coerce_numeric_vec(params$horizons_micro))
turnover_K_list <- as.integer(coerce_numeric_vec(params$turnover_K_list))
turnover_horizons <- as.integer(coerce_numeric_vec(params$turnover_horizons))

if (is.na(K_cut_target) || is.na(K_tail_buffer) || is.na(Smoothing_h)) {
  stop(
    "K_cut_target, K_tail_buffer, and smoothing_h must be numeric. Got: ",
    "K_cut_target=", params$K_cut_target, ", ",
    "K_tail_buffer=", params$K_tail_buffer, ", ",
    "smoothing_h=", params$smoothing_h
  )
}

if (Smoothing_h < 0) {
  stop("smoothing_h must be >= 0. Got: ", params$smoothing_h)
}

if (is.na(bootstrap_B) || bootstrap_B < 1) {
  stop("bootstrap_B must be a positive integer. Got: ", params$bootstrap_B)
}

if (is.na(bootstrap_k_min) || bootstrap_k_min < 1) {
  stop("bootstrap_k_min must be a positive integer. Got: ", params$bootstrap_k_min)
}

if (is.na(sim_T_weeks) || sim_T_weeks < 1) {
  stop("sim_T_weeks must be a positive integer. Got: ", params$sim_T_weeks)
}

if (is.na(sim_n_paths) || sim_n_paths < 1) {
  stop("sim_n_paths must be a positive integer. Got: ", params$sim_n_paths)
}

if (is.na(sim_mu) || is.na(sim_sigma) || is.na(sim_entry_frac)) {
  stop(
    "sim_mu, sim_sigma, and sim_entry_frac must be numeric. Got: ",
    "sim_mu=", params$sim_mu, ", ",
    "sim_sigma=", params$sim_sigma, ", ",
    "sim_entry_frac=", params$sim_entry_frac
  )
}

if (sim_entry_frac < 0 || sim_entry_frac > 1) {
  stop("sim_entry_frac must be in [0, 1]. Got: ", params$sim_entry_frac)
}

if (is.na(sim_K_xi) || sim_K_xi < 2) {
  stop("sim_K_xi must be an integer >= 2. Got: ", params$sim_K_xi)
}

horizons_growth <- horizons_growth[!is.na(horizons_growth) & horizons_growth >= 1L]
horizons_durable <- horizons_durable[!is.na(horizons_durable) & horizons_durable >= 1L]
horizons_micro <- horizons_micro[!is.na(horizons_micro) & horizons_micro >= 1L]

```

```

turnover_K_list <- turnover_K_list[!is.na(turnover_K_list) & turnover_K_list >= 1L]
turnover_horizons <- turnover_horizons[!is.na(turnover_horizons) & turnover_horizons >= 1L]

if (length(horizons_growth) == 0 || length(horizons_durable) == 0 || length(horizons_micro) == 0) {
  stop(
    "horizons_growth, horizons_durable, and horizons_micro must include at least one integer >= 1."
  )
}

if (length(turnover_K_list) == 0 || length(turnover_horizons) == 0) {
  stop(
    "turnover_K_list and turnover_horizons must include at least one integer >= 1."
  )
}

max_h_durable <- max(horizons_durable)
if (sim_T_weeks < max_h_durable) {
  message(
    "sim_T_weeks (", sim_T_weeks, ") < max(horizons_durable) (", max_h_durable,
    "); using sim_T_weeks=", max_h_durable, "."
  )
  sim_T_weeks <- max_h_durable
}

```

Load weekly Facebook data and construct core panel

The exploration PDFs treat the weekly Facebook file as weekly snapshots with (at least): `date`, `endpoint_id`, `metric_value`, `rank` (rank is recomputed defensively below).

We construct:

- `share_global`: each endpoint's share of total weekly attention (sums to 1 within week).
- `share_topK`: same share, renormalized within the top-K (`K_cut`) each week.

```

raw <- read_parquet(fb_weekly_parquet_path)

endpoint_weekly <- raw %>%
  transmute(
    week = as.Date(date),
    endpoint_id,
    metric_value,
    rank = rank
  ) %>%
  group_by(week) %>%
  arrange(desc(metric_value), .by_group = TRUE) %>%
  mutate(
    rank = row_number(),
    share_global = metric_value / sum(metric_value, na.rm = TRUE)
  ) %>%
  ungroup()

max_rank_by_week <- endpoint_weekly %>%
  group_by(week) %>%
  summarise(max_rank = max(rank), .groups = "drop")

```

```

max_rank_seen <- max(max_rank_by_week$max_rank, na.rm = TRUE)

K_cut <- min(as.integer(K_cut_target), as.integer(max_rank_seen))
K_max <- min(as.integer(K_cut + K_tail_buffer), as.integer(max_rank_seen))

if (sim_K_xi > K_cut) {
  message("sim_K_xi (", sim_K_xi, ") > K_cut (", K_cut, "); using sim_K_xi=", K_cut, ".")
  sim_K_xi <- K_cut
}

endpoint_weekly <- endpoint_weekly %>%
  group_by(week) %>%
  mutate(
    share_topK = if_else(
      rank <= K_cut,
      share_global / sum(share_global[rank <= K_cut], na.rm = TRUE),
      NA_real_
    )
  ) %>%
  ungroup()

glimpse(endpoint_weekly)

## Rows: 2,391,203
## Columns: 6
## $ week          <date> 2020-10-26, 2020-10-26, 2020-10-26, 2020-10-26, 2020-10-26
## $ endpoint_id   <chr> "Fox News", "Dan Bongino", "9GAG", "Breitbart", "LADbible"
## $ metric_value  <int> 10970437, 7638188, 6693697, 6625121, 6027168, 5725863, 54~
## $ rank          <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17~
## $ share_global  <dbl> 0.014472444, 0.010076467, 0.008830473, 0.008740006, 0.007~
## $ share_topK    <dbl> 0.014517286, 0.010107689, 0.008857835, 0.008767087, 0.007~

```

Data QA and coverage

This reproduces the “weekly coverage & scale checks” used in v6.

```

endpoint_weekly %>%
  summarise(
    min_week = min(week),
    max_week = max(week),
    n_weeks = n_distinct(week),
    n_endpoints = n_distinct(endpoint_id),
    max_rank_seen = max(rank)
  )

## # A tibble: 1 x 5
##   min_week   max_week   n_weeks n_endpoints max_rank_seen
##   <date>     <date>     <int>     <int>         <int>
## 1 2020-10-26 2024-03-04     176     15685         14714

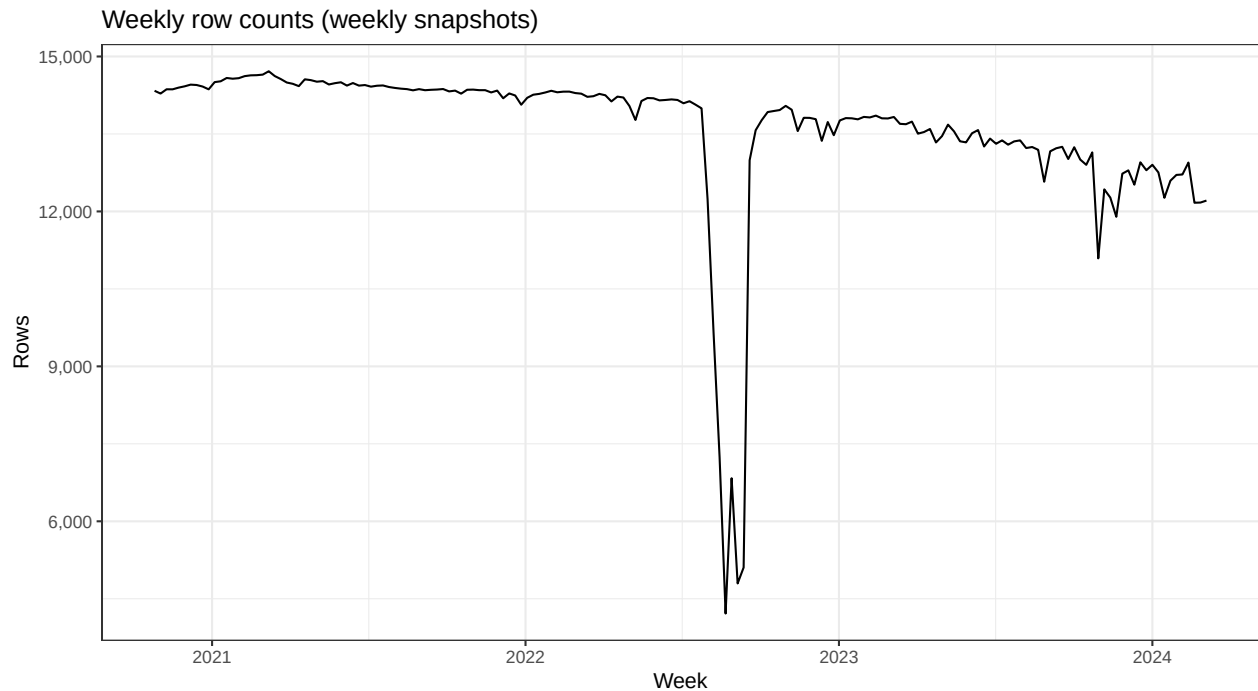
weekly_totals <- endpoint_weekly %>%
  count(week, name = "n_rows") %>%
  arrange(week)

summary(weekly_totals$n_rows)

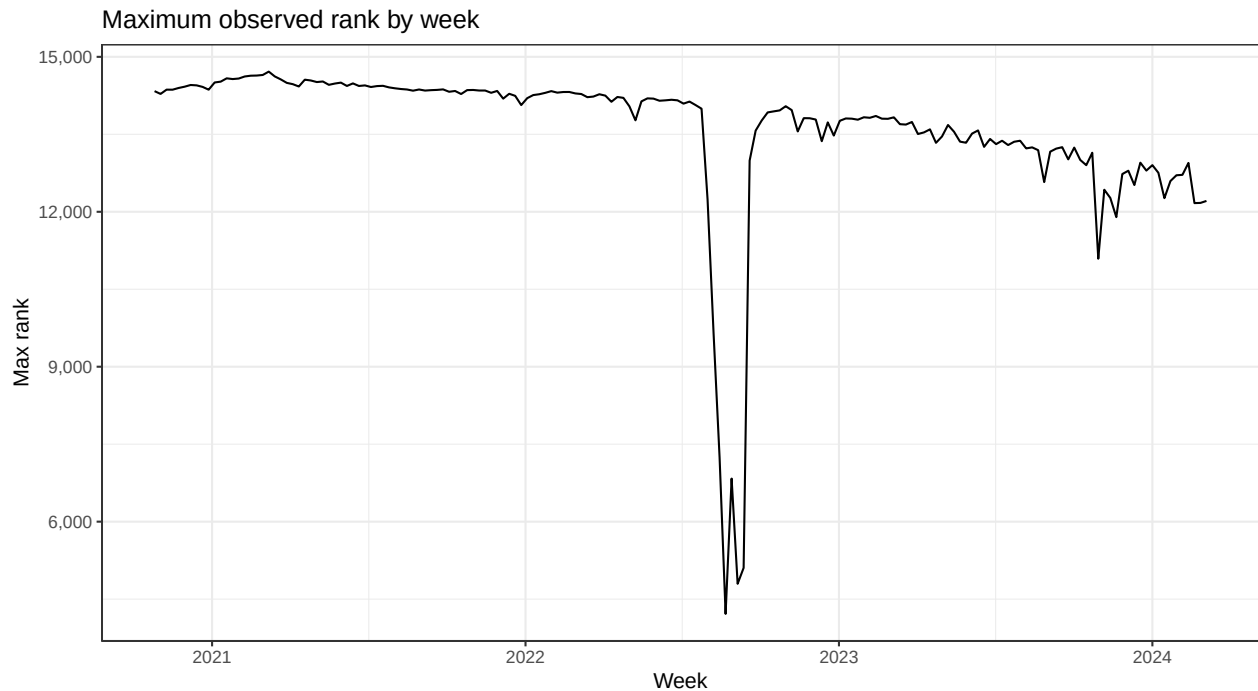
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      4214   13351   14066   13586   14359   14714
```

```
ggplot(weekly_totals, aes(week, n_rows)) +
  geom_line() +
  scale_y_continuous(labels = comma) +
  labs(
    title = "Weekly row counts (weekly snapshots)",
    x = "Week",
    y = "Rows"
  ) +
  theme_bw()
```



```
ggplot(max_rank_by_week, aes(week, max_rank)) +
  geom_line() +
  scale_y_continuous(labels = comma) +
  labs(
    title = "Maximum observed rank by week",
    x = "Week",
    y = "Max rank"
  ) +
  theme_bw()
```

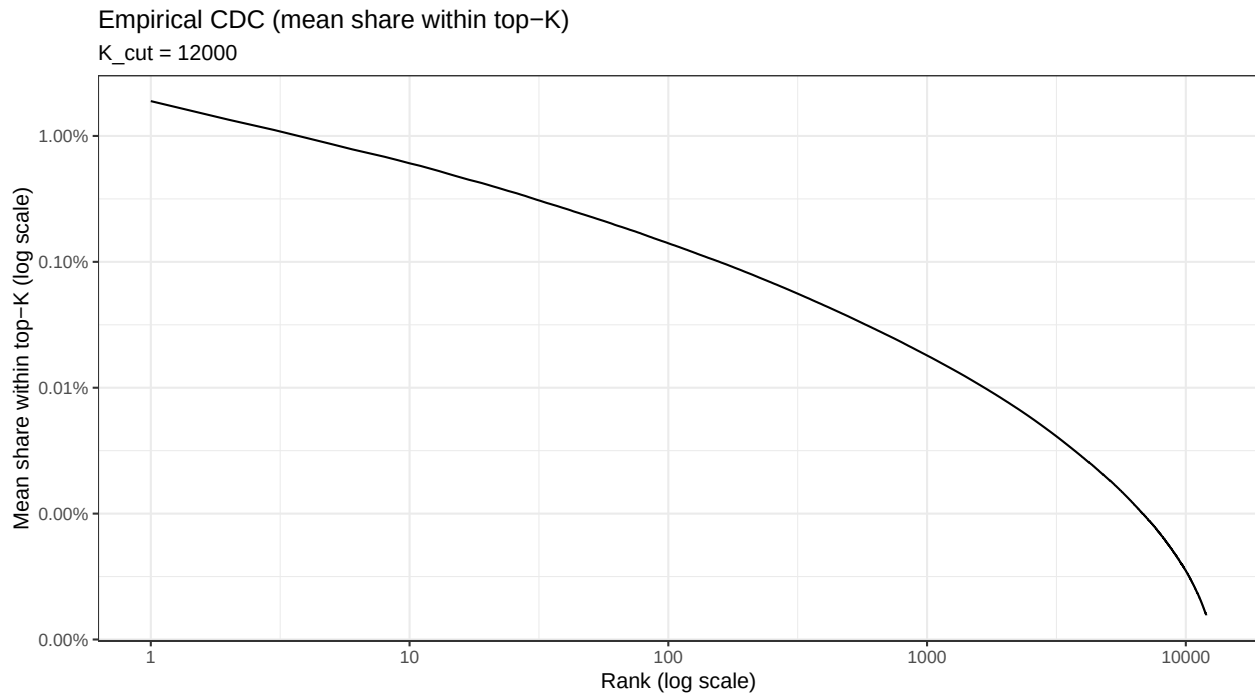


Stylized fact A: Capital Distribution Curve (CDC) and stability

The empirical CDC is the mean (across weeks) of `share_topK` at each rank within top-K.

```
emp_cdc <- endpoint_weekly %>%
  filter(rank <= K_cut) %>%
  group_by(rank) %>%
  summarise(w_bar = mean(share_topK, na.rm = TRUE), .groups = "drop") %>%
  arrange(rank)

ggplot(emp_cdc, aes(rank, w_bar)) +
  geom_line() +
  scale_x_log10() +
  scale_y_log10(labels = percent_format(accuracy = 0.01)) +
  labs(
    title = "Empirical CDC (mean share within top-K)",
    subtitle = paste0("K_cut = ", K_cut),
    x = "Rank (log scale)",
    y = "Mean share within top-K (log scale)"
  ) +
  theme_bw()
```



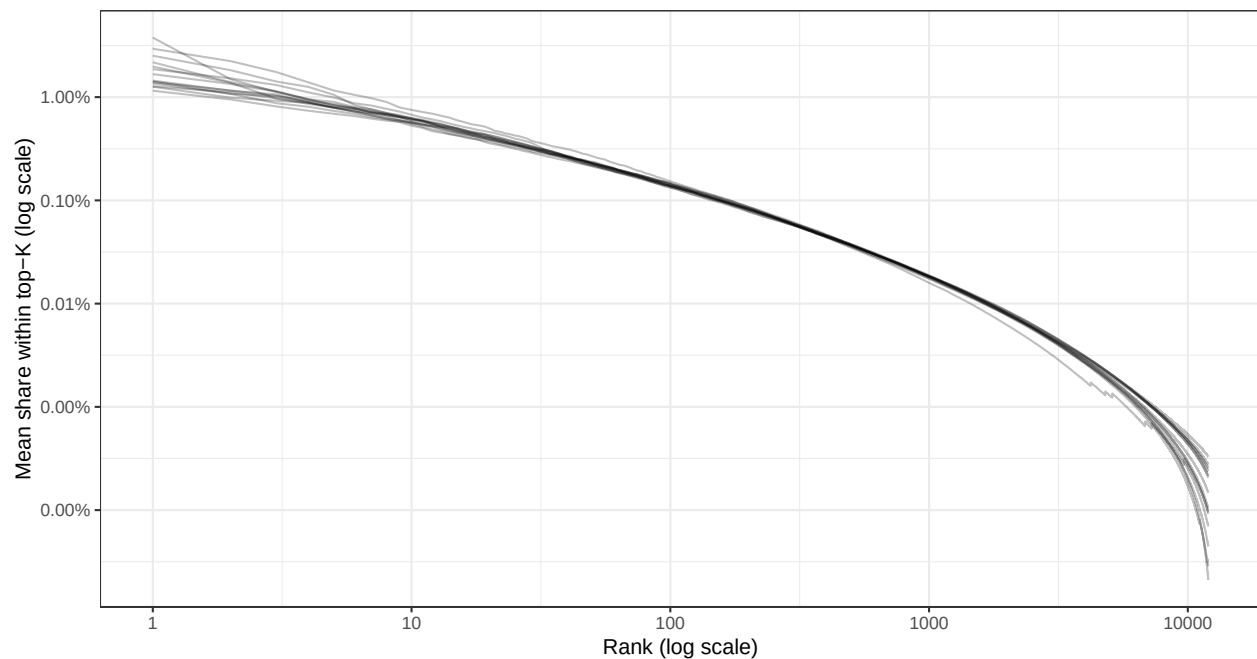
CDC stability over time windows

A simple stability check: compute period-level CDCs and compare to the full-sample CDC.

```
cdc_by_period <- endpoint_weekly %>%
  filter(rank <= K_cut) %>%
  mutate(period = floor_date(week, "quarter")) %>%
  group_by(period, rank) %>%
  summarise(w_bar = mean(share_topK, na.rm = TRUE), .groups = "drop")

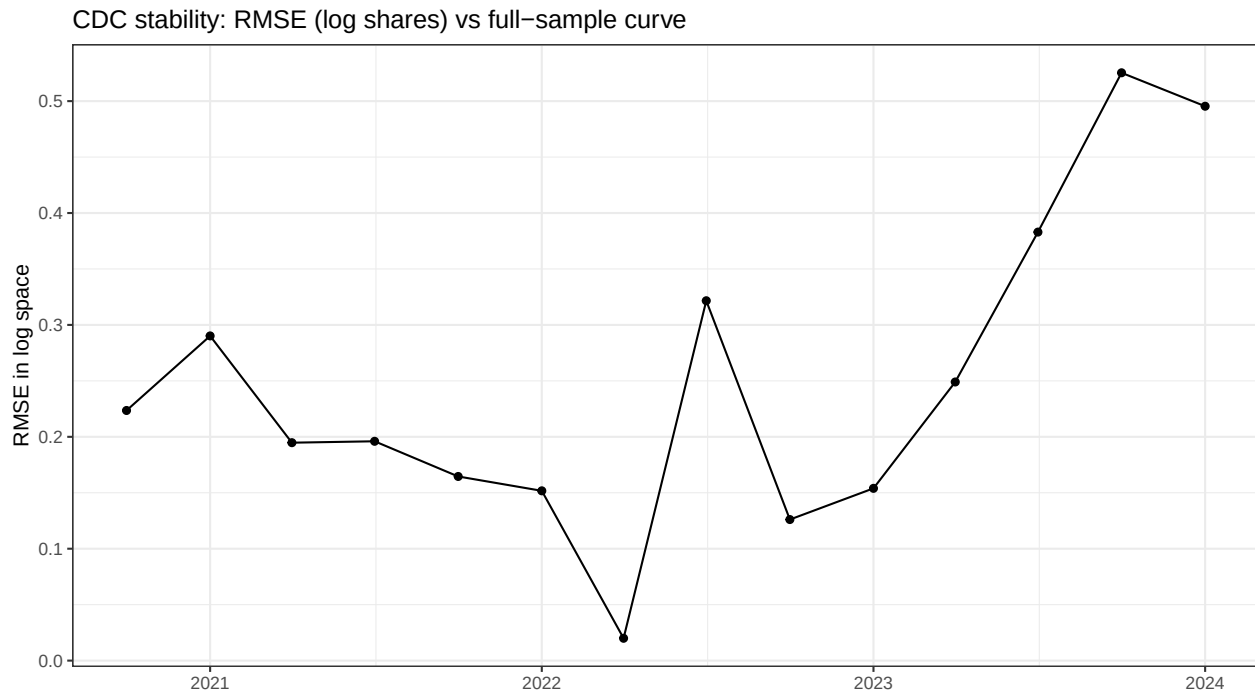
ggplot(cdc_by_period, aes(rank, w_bar, group = period)) +
  geom_line(alpha = 0.25) +
  scale_x_log10() +
  scale_y_log10(labels = percent_format(accuracy = 0.01)) +
  labs(
    title = "CDC curves by quarter (visual stability check)",
    x = "Rank (log scale)",
    y = "Mean share within top-K (log scale)"
  ) +
  theme_bw()
```

CDC curves by quarter (visual stability check)



```
cdc_rmse_vs_full <- cdc_by_period %>%
  inner_join(emp_cdc, by = "rank", suffix = c("_period", "_full")) %>%
  group_by(period) %>%
  summarise(
    rmse_log = sqrt(mean((log(w_bar_period) - log(w_bar_full))^2, na.rm = TRUE)),
    .groups = "drop"
  )

ggplot(cdc_rmse_vs_full, aes(period, rmse_log)) +
  geom_line() +
  geom_point(size = 1.5) +
  labs(
    title = "CDC stability: RMSE (log shares) vs full-sample curve",
    x = NULL,
    y = "RMSE in log space"
  ) +
  theme_bw()
```



Endpoint-level forward growth distributions (v2/v3)

This reproduces the early v2/v3 exploratory step: compute forward-looking log growth in *global share* for individual endpoints over multiple horizons, and compare distributions across rank buckets.

```
rank_bucket_simple <- function(r) {
  case_when(
    r <= 25 ~ "large",
    r <= 250 ~ "midsize",
    TRUE ~ "small"
  )
}

endpoint_base <- endpoint_weekly %>%
  transmute(
    week,
    endpoint_id,
    rank,
    bucket = rank_bucket_simple(rank),
    log_share = log(pmax(share_global, 1e-12))
  )

growth_long <- purrr::map_dfr(horizons_growth, function(h) {
  endpoint_base %>%
    mutate(
      horizon_weeks = as.integer(h),
      week_tp = week + lubridate::weeks(as.integer(h))
    ) %>%
    left_join(
      endpoint_base %>%
        transmute(endpoint_id, week_tp = week, log_share_tp = log_share),
      by = c("endpoint_id", "week_tp")
    )
})
```

```

    ) %>%
    mutate(dlog_share = log_share_tp - log_share) %>%
    filter(is.finite(dlog_share))
  })

```

```

growth_summ <- growth_long %>%
  group_by(bucket, horizon_weeks) %>%
  summarise(
    n = n(),
    med = median(dlog_share, na.rm = TRUE),
    p10 = quantile(dlog_share, 0.10, na.rm = TRUE),
    p90 = quantile(dlog_share, 0.90, na.rm = TRUE),
    .groups = "drop"
  )

```

```
growth_summ
```

```
## # A tibble: 9 x 6
##   bucket horizon_weeks      n      med  p10  p90
##   <chr>          <int> <int>   <dbl> <dbl> <dbl>
## 1 large             1  4344 -0.126  -1.16 0.273
## 2 large             7  4191 -0.283  -2.14 0.340
## 3 large            28  3643 -0.529  -2.56 0.371
## 4 midsize           1 39028 -0.147  -1.36 0.381
## 5 midsize           7 37330 -0.299  -2.05 0.462
## 6 midsize          28 32274 -0.532  -2.72 0.501
## 7 small             1 2265644 -0.00189 -1.02 1.03
## 8 small             7 2134069 -0.00664 -1.30 1.28
## 9 small            28 1829745 -0.0518  -1.69 1.50

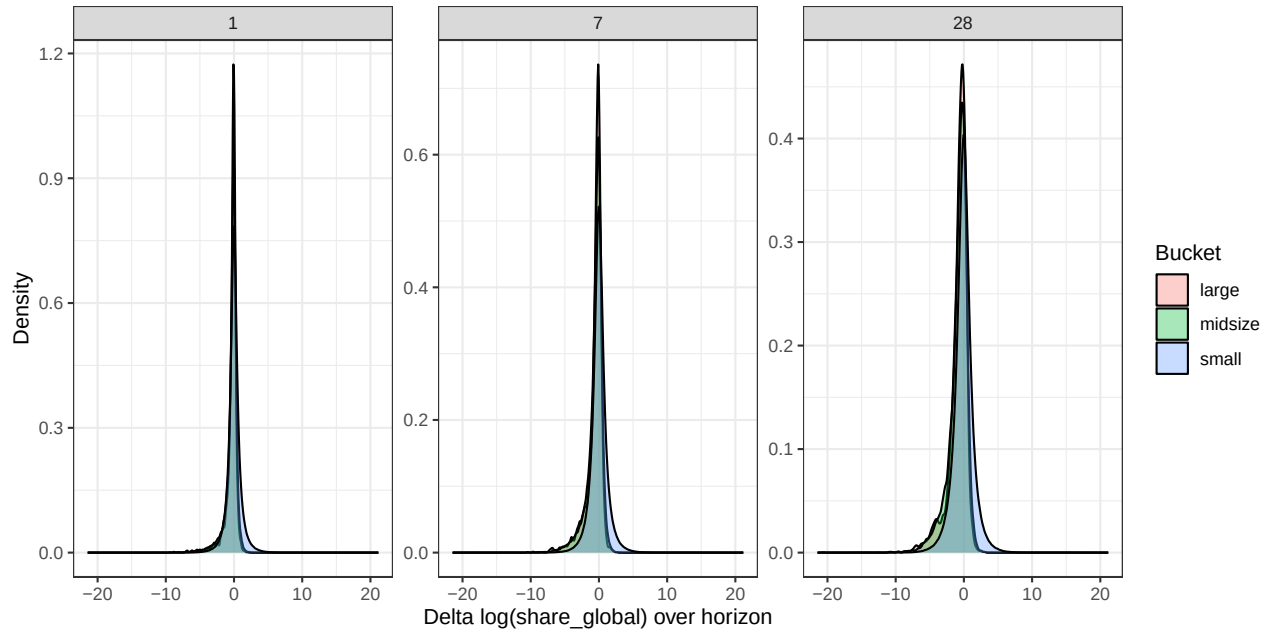
```

```

ggplot(growth_long, aes(x = dlog_share, fill = bucket)) +
  geom_density(alpha = 0.35) +
  facet_wrap(~ horizon_weeks, scales = "free_y") +
  labs(
    title = "Forward log-growth in global share by rank bucket",
    subtitle = "Endpoint-level growth distributions (identity-based), as in v2/v3",
    x = "Delta log(share_global) over horizon",
    y = "Density",
    fill = "Bucket"
  ) +
  theme_bw()

```

Forward log-growth in global share by rank bucket
 Endpoint-level growth distributions (identity-based), as in v2/v3



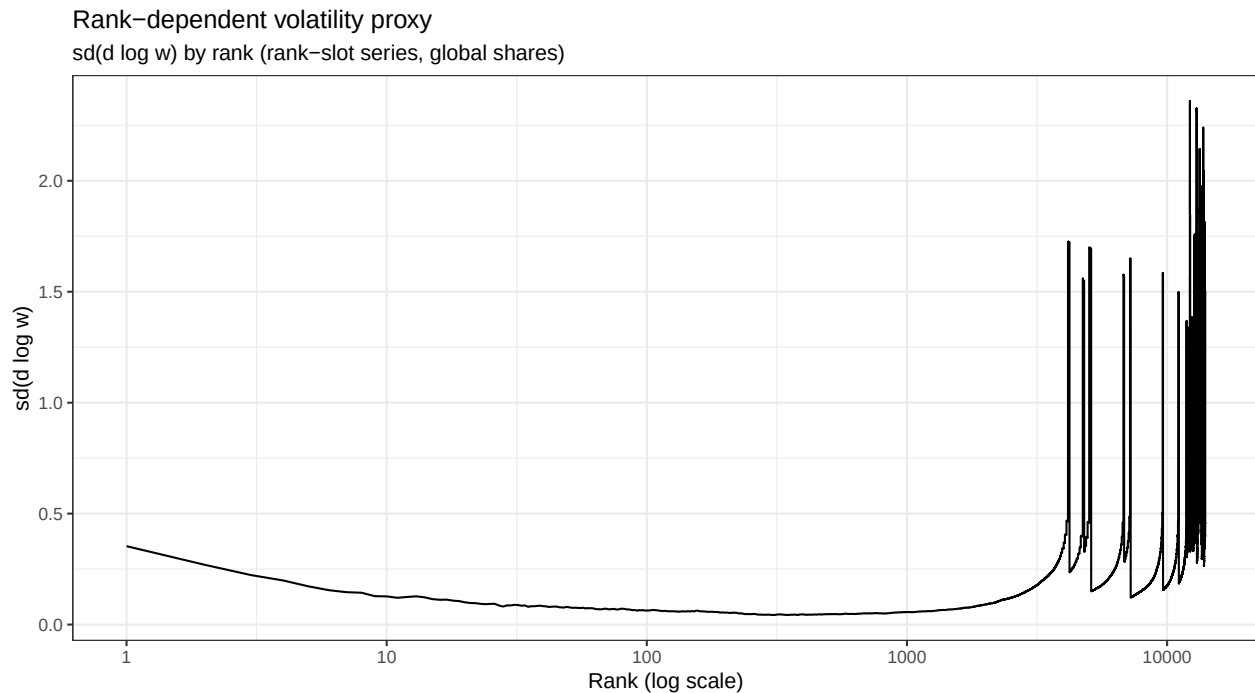
Stylized fact B: Rank-dependent drift and volatility (rank-slot increments)

We work with rank-slot series: for each rank k , the sequence of shares of the endpoint occupying that rank each week. This is the object that rank-based diffusion models target.

```
rank_panel <- endpoint_weekly %>%
  group_by(rank) %>%
  arrange(week, .by_group = TRUE) %>%
  mutate(
    share_global = pmax(share_global, 1e-12),
    log_w = log(share_global),
    dlogw = log_w - lag(log_w)
  ) %>%
  ungroup() %>%
  filter(is.finite(dlogw))

gauss_params_raw <- rank_panel %>%
  filter(rank <= K_max) %>%
  group_by(rank) %>%
  summarise(
    mean_dlogw = mean(dlogw, na.rm = TRUE),
    sd_dlogw = sd(dlogw, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  right_join(tibble(rank = 1:K_max), by = "rank") %>%
  arrange(rank) %>%
  mutate(
    mean_dlogw = replace_na(mean_dlogw, 0),
    sd_dlogw = replace_na(sd_dlogw, 0),
    sd_dlogw = if_else(sd_dlogw <= 0 | !is.finite(sd_dlogw), 1e-6, sd_dlogw)
  )
```

```
ggplot(gauss_params_raw, aes(rank, sd_dlogw)) +
  geom_line() +
  scale_x_continuous(trans = "log10") +
  labs(
    title = "Rank-dependent volatility proxy",
    subtitle = "sd(d log w) by rank (rank-slot series, global shares)",
    x = "Rank (log scale)",
    y = "sd(d log w)"
  ) +
  theme_bw()
```



Smoothed per-rank moments (v9+)

```
moving_average_rank <- function(x, h) {
  stopifnot(h >= 0L)
  n <- length(x)
  if (h == 0L) return(x)
  cs <- c(0, cumsum(replace_na(x, 0)))
  out <- numeric(n)
  for (r in seq_len(n)) {
    lo <- max(1L, r - h)
    hi <- min(n, r + h)
    out[r] <- (cs[hi + 1L] - cs[lo]) / (hi - lo + 1L)
  }
  out
}

build_gaussian_params_smoothed <- function(h) {
  m <- gauss_params_raw$mean_dlogw
  s <- gauss_params_raw$sd_dlogw
  tibble(
```

```

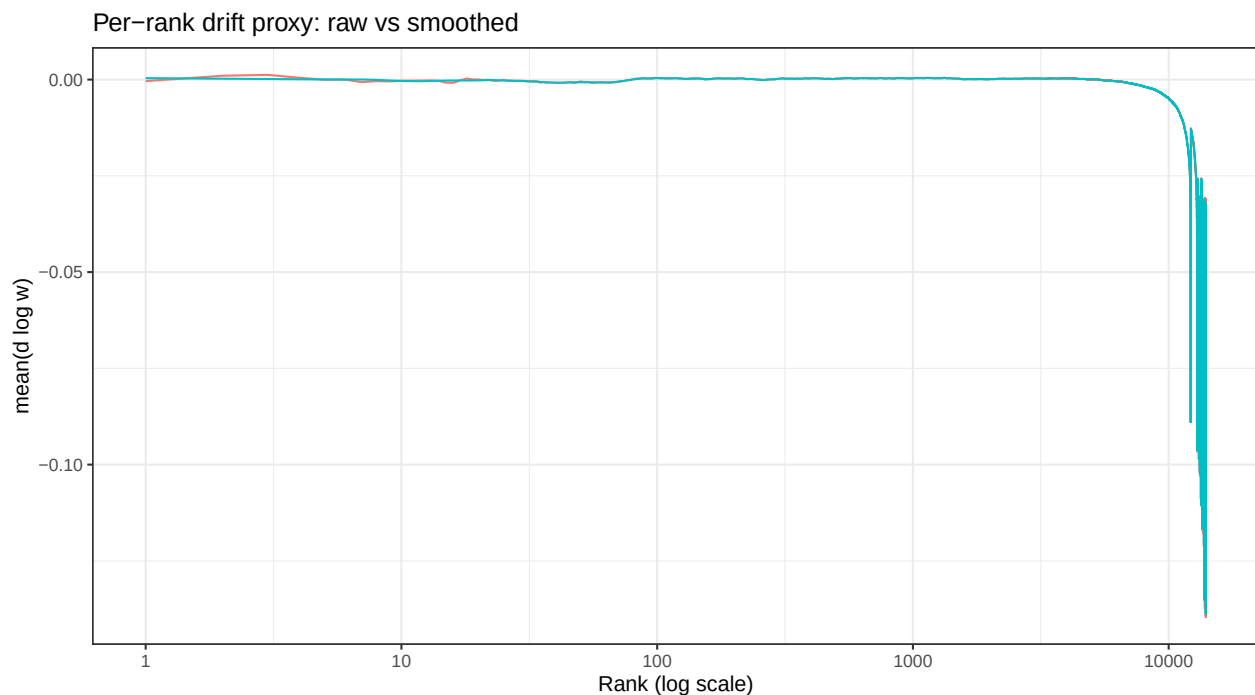
    rank = 1:K_max,
    mean_dlogw_s = moving_average_rank(m, h = h),
    sd_dlogw_s = pmax(moving_average_rank(s, h = h), 1e-6)
  )
}

sm_params <- build_gaussian_params_smoothed(h = as.integer(Smoothing_h))

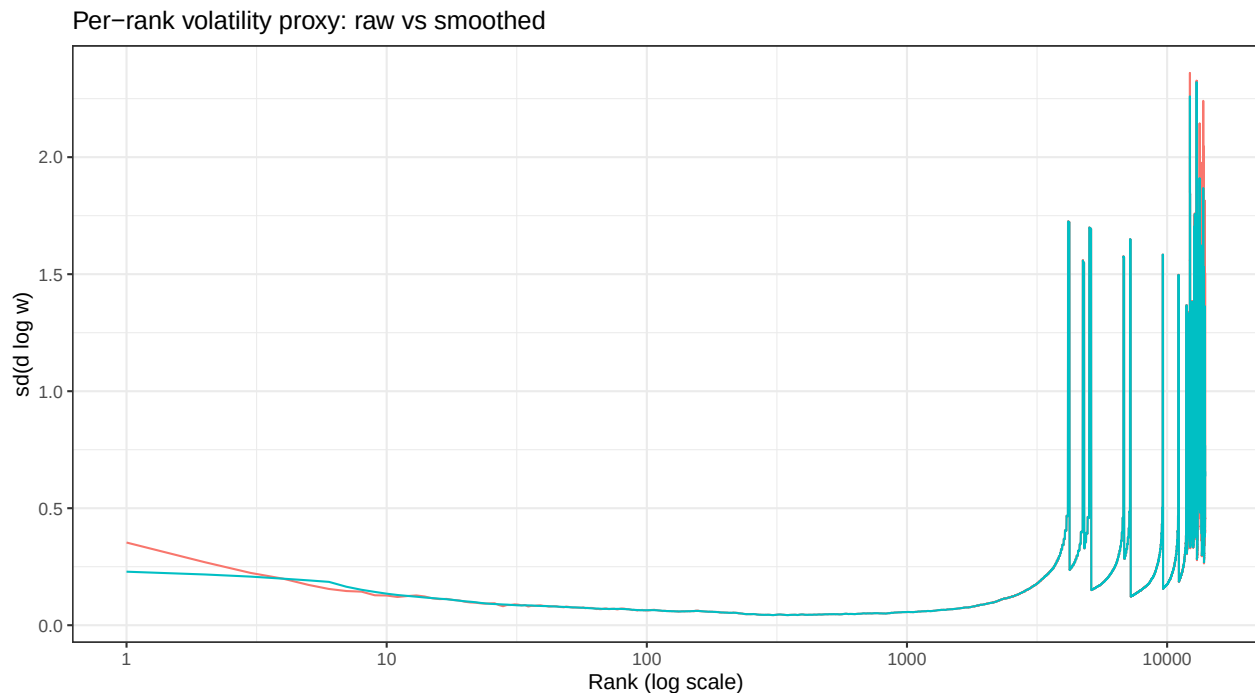
sm_plot <- sm_params %>%
  select(rank, mean_dlogw_s, sd_dlogw_s) %>%
  left_join(gauss_params_raw %>% select(rank, mean_dlogw, sd_dlogw), by = "rank") %>%
  pivot_longer(
    cols = c(mean_dlogw, mean_dlogw_s, sd_dlogw, sd_dlogw_s),
    names_to = "series",
    values_to = "value"
  ) %>%
  mutate(kind = case_when(
    str_detect(series, "^mean") ~ "mean",
    TRUE ~ "sd"
  ))

ggplot(filter(sm_plot, kind == "mean"), aes(rank, value, color = series)) +
  geom_line() +
  scale_x_continuous(trans = "log10") +
  labs(
    title = "Per-rank drift proxy: raw vs smoothed",
    x = "Rank (log scale)",
    y = "mean(d log w)"
  ) +
  theme_bw() +
  guides(color = "none")

```



```
ggplot(filter(sm_plot, kind == "sd"), aes(rank, value, color = series)) +
  geom_line() +
  scale_x_continuous(trans = "log10") +
  labs(
    title = "Per-rank volatility proxy: raw vs smoothed",
    x = "Rank (log scale)",
    y = "sd(d log w)"
  ) +
  theme_bw() +
  guides(color = "none")
```



Stylized fact C: Durable change targets (v7–v10)

These targets summarize the typical magnitude of change over horizons (default 4 and 8 weeks), separately by rank buckets.

```
bucket_def <- list(
  breaks = params$bucket_breaks,
  labels = params$bucket_labels
)

assign_bucket <- function(rank, bucket_def) {
  cut(
    rank,
    breaks = bucket_def$breaks,
    labels = bucket_def$labels,
    include.lowest = TRUE,
    right = TRUE
  )
}

compute_emp_endpoint_change <- function(df, horizons, bucket_def) {
```

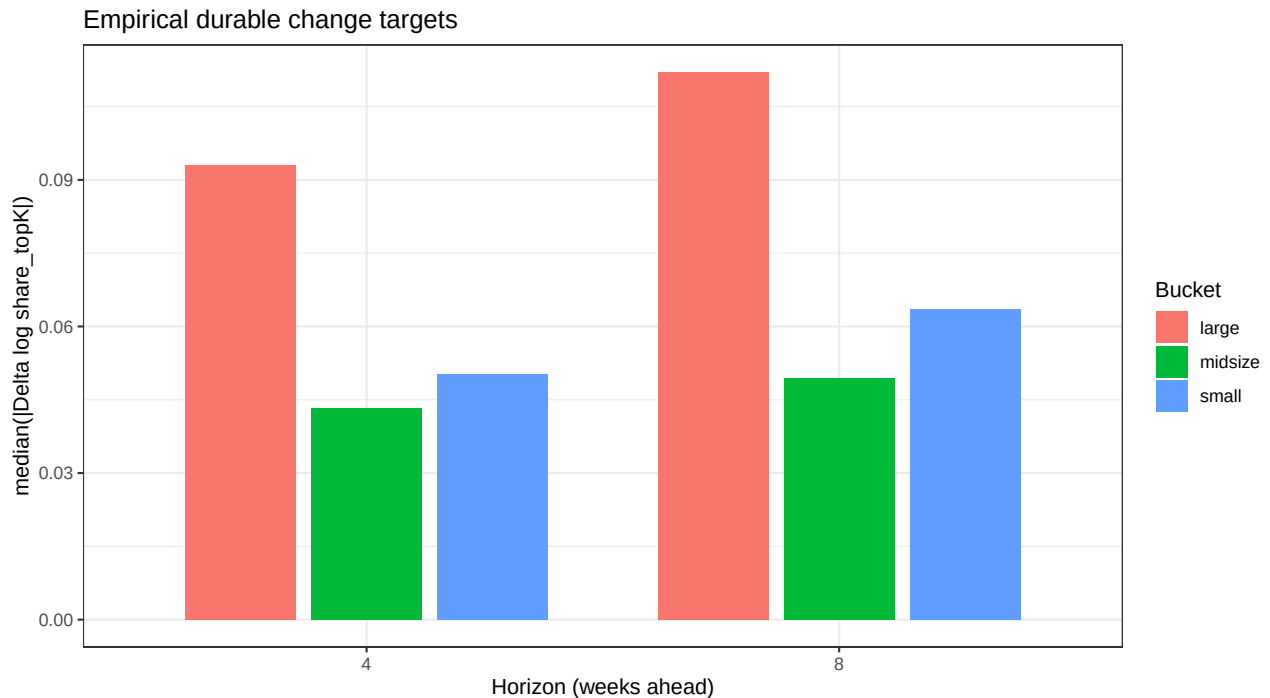
```

df2 <- df %>%
  group_by(rank) %>%
  arrange(week, .by_group = TRUE) %>%
  mutate(log_w = log(pmax(share_topK, 1e-12)))
purrr::map_dfr(horizons, function(h) {
  df2 %>%
    mutate(d = dplyr::lead(log_w, h) - log_w) %>%
    ungroup() %>%
    mutate(
      horizon = as.integer(h),
      bucket = assign_bucket(rank, bucket_def)
    ) %>%
    filter(!is.na(d), !is.na(bucket)) %>%
    group_by(bucket, horizon) %>%
    summarise(
      med_abs_emp = median(abs(d), na.rm = TRUE),
      .groups = "drop"
    )
})
}

emp_targets <- compute_emp_endpoint_change(
  df = endpoint_weekly %>% filter(rank <= K_cut),
  horizons = horizons_durable,
  bucket_def = bucket_def
)

ggplot(emp_targets, aes(x = factor(horizon), y = med_abs_emp, fill = bucket)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7) +
  labs(
    title = "Empirical durable change targets",
    x = "Horizon (weeks ahead)",
    y = "median(|Delta log share_topK|)",
    fill = "Bucket"
  ) +
  theme_bw()

```



Martingale-style diagnostic (v7)

A simple one-step regression: $\text{dlogw}(t+1)$ on $\text{dlogw}(t)$ for rank-slot series, summarized by bucket.

```
diag_df <- rank_panel %>%
  group_by(rank) %>%
  arrange(week, .by_group = TRUE) %>%
  mutate(
    dlogw_t = dlogw,
    dlogw_t1 = lead(dlogw_t, 1)
  ) %>%
  ungroup() %>%
  mutate(bucket = assign_bucket(rank, bucket_def)) %>%
  filter(is.finite(dlogw_t), is.finite(dlogw_t1), !is.na(bucket))

martingale_results <- diag_df %>%
  group_by(bucket) %>%
  group_modify(~ {
    fit <- lm(dlogw_t1 ~ dlogw_t, data = .x)
    broom::tidy(fit)
  }) %>%
  ungroup()

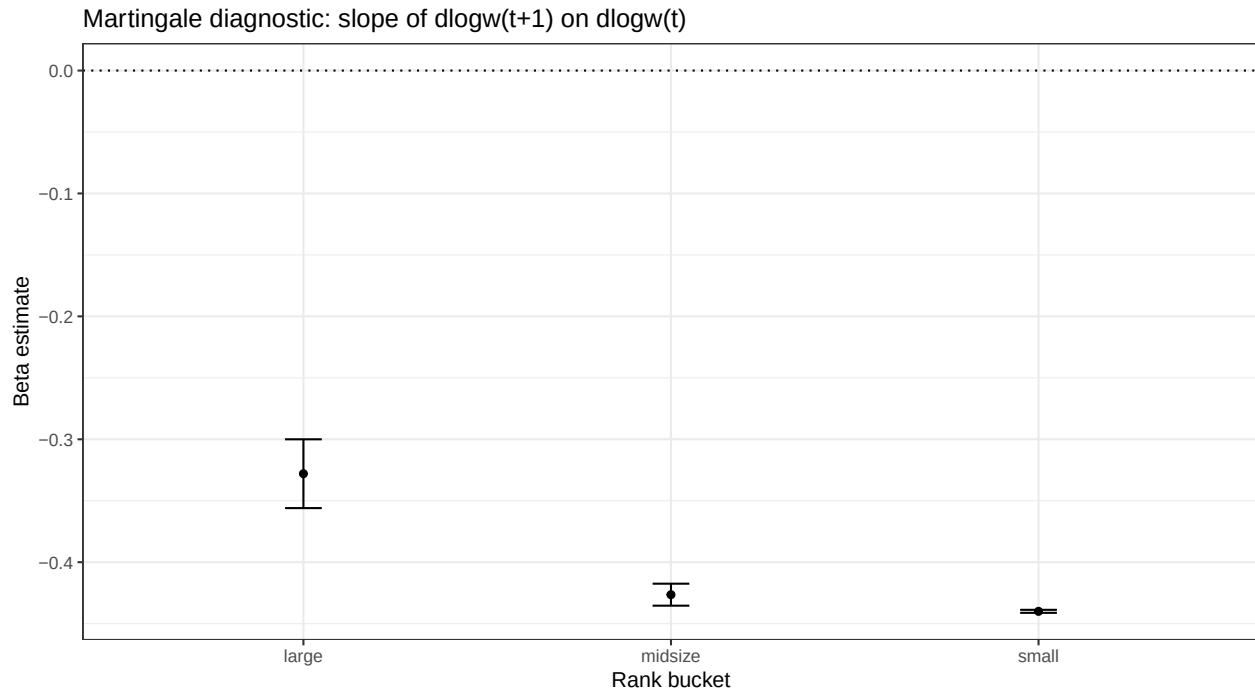
ggplot(martingale_results %>% filter(term == "dlogw_t"), aes(bucket, estimate)) +
  geom_point() +
  geom_errorbar(
    aes(ymin = estimate - 1.96 * std.error, ymax = estimate + 1.96 * std.error),
    width = 0.1
  ) +
  geom_hline(yintercept = 0, linetype = 3) +
  labs(
    title = "Martingale diagnostic: slope of dlogw(t+1) on dlogw(t)",

```

```

x = "Rank bucket",
y = "Beta estimate"
) +
theme_bw()

```



Stylized fact D: Gap structure via $\text{Xi}_k(t) = \log(w_k / w_{k+1})$ (v11)

We compute Xi using *global* shares, with ranks recomputed within each week.

```

xi_weekly <- endpoint_weekly %>%
  group_by(week) %>%
  arrange(rank, .by_group = TRUE) %>%
  mutate(
    w = pmax(share_global, 1e-18),
    w_next = lead(w),
    xi = log(w / w_next),
    k = rank
  ) %>%
  ungroup() %>%
  filter(is.finite(xi), is.finite(k))

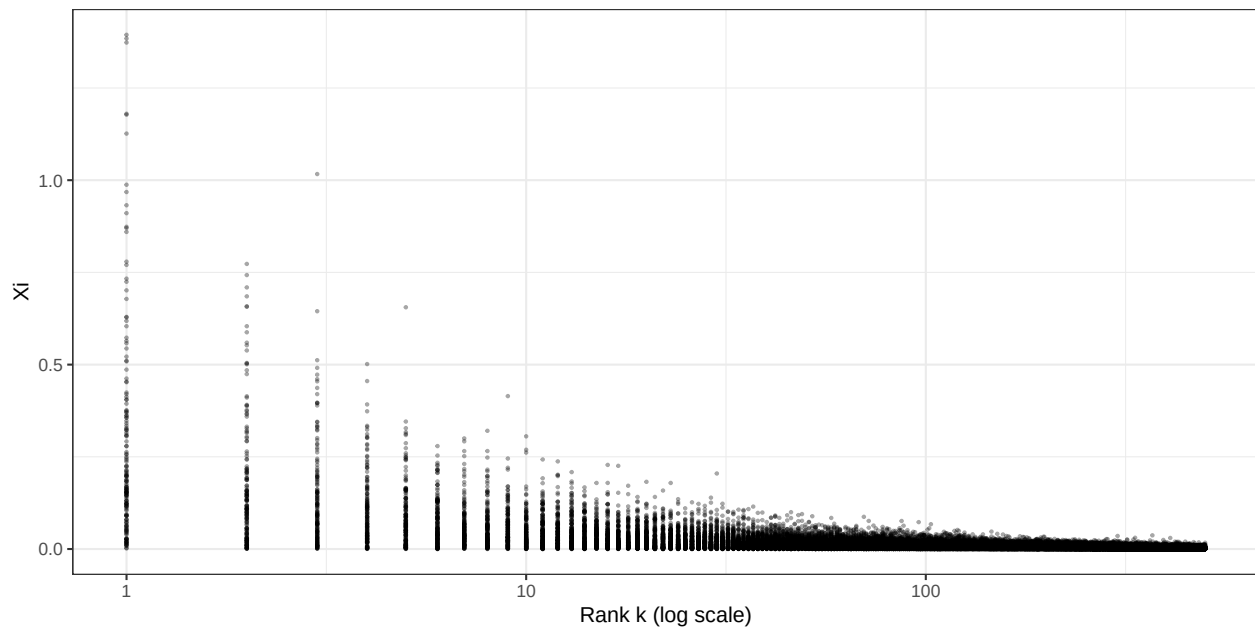
K_xi_est <- min(2000L, K_cut)

# scatter (restricted for clarity)
ggplot(filter(xi_weekly, k <= 500L), aes(x = k, y = xi)) +
  geom_point(alpha = 0.35, size = 0.4) +
  scale_x_continuous(trans = "log10") +
  labs(
    title = "Log gaps (Xi) by rank:  $\text{Xi}_k(t) = \log(w_k / w_{k+1})$ ",
    subtitle = "Restricted to top 500 ranks for visibility",
    x = "Rank k (log scale)",
    y = "Xi"
  )

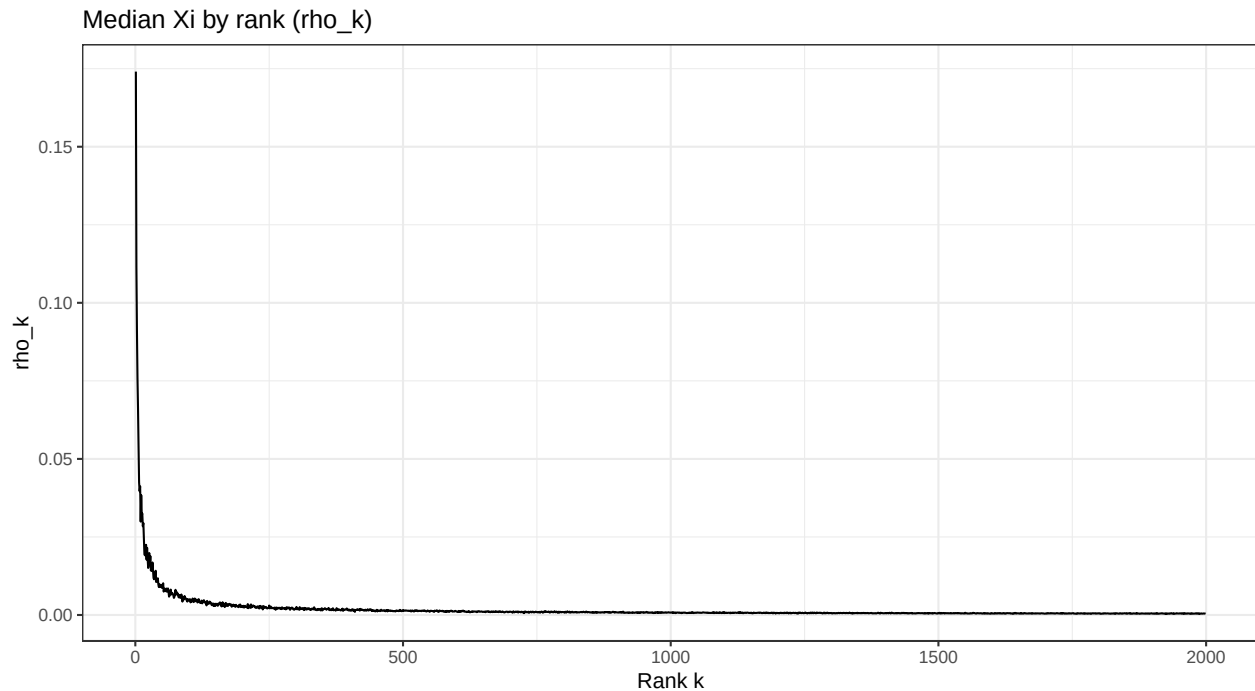
```

```
) +  
theme_bw()
```

Log gaps (Xi) by rank: $\text{Xi}_k(t) = \log(w_k / w_{k+1})$
 Restricted to top 500 ranks for visibility



```
rho_by_k_med <- xi_weekly %>%  
  filter(k <= (K_xi_est - 1L)) %>%  
  group_by(k) %>%  
  summarise(rho_k = median(xi, na.rm = TRUE), .groups = "drop")  
  
ggplot(rho_by_k_med, aes(k, rho_k)) +  
  geom_line() +  
  theme_bw() +  
  labs(  
    title = "Median Xi by rank (rho_k)",  
    x = "Rank k",  
    y = "rho_k"  
  )
```



v11-style $\sigma_{\text{hat}}(k)$ construction

Following v11:

- Let g_k be smoothed per-rank drift proxy (mean of $d \log w$) for ranks.
- Define $S_k = \text{cumsum}(g_k)$.
- Combine with ρ_k to form $\sigma_{\text{hat}}(k) = \sqrt{-2 S_k \rho_k}$ when positive.

```
g_by_k <- sm_params %>%
  transmute(k = rank, g_k = mean_dlogw_s) %>%
  filter(k <= (K_xi_est + 1L)) %>%
  arrange(k)

S_by_k <- g_by_k %>%
  mutate(S_k = cumsum(g_k))

sigma_hat_by_k <- rho_by_k_med %>%
  inner_join(S_by_k %>% select(k, S_k), by = "k") %>%
  mutate(
    sigma2_hat = -2 * S_k * rho_k,
    sigma_hat = if_else(sigma2_hat > 0, sqrt(sigma2_hat), NA_real_)
  )

sigma_hat_by_k %>%
  summarise(med_sigma = median(sigma_hat, na.rm = TRUE))

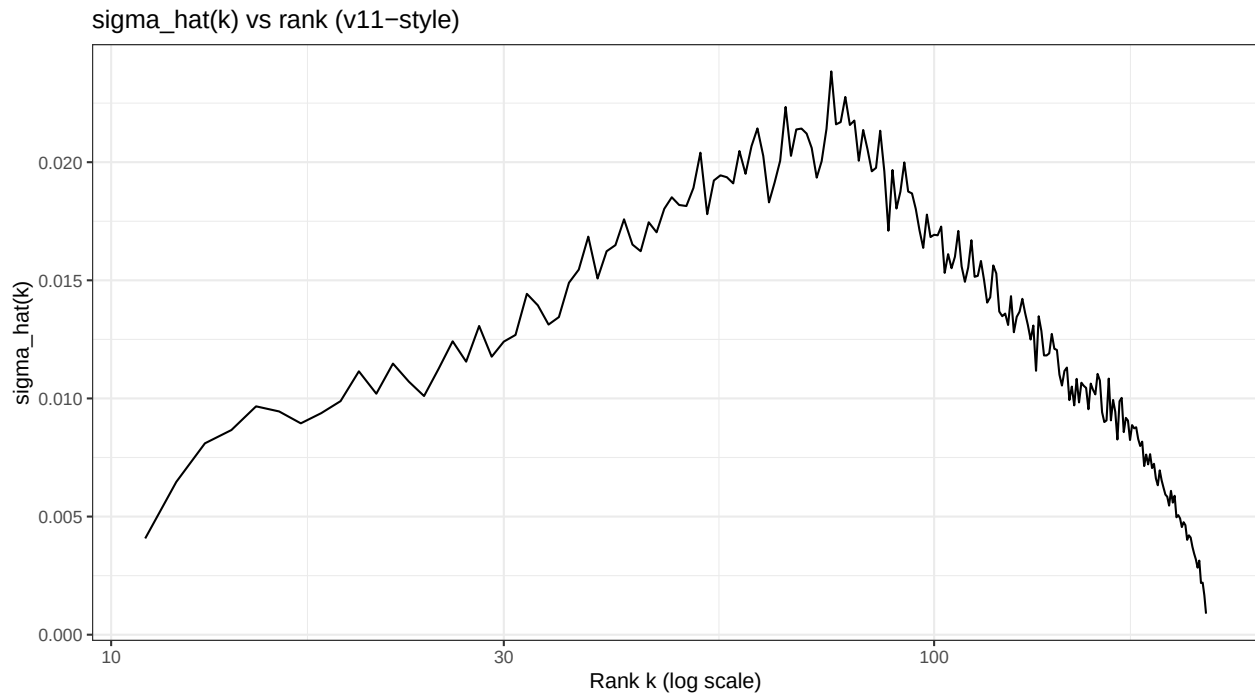
## # A tibble: 1 x 1
##   med_sigma
##   <dbl>
## 1      0.0128

ggplot(
  filter(sigma_hat_by_k, k >= 10L, !is.na(sigma_hat)),
```

```

aes(x = k, y = sigma_hat)
) +
geom_line() +
scale_x_continuous(trans = "log10") +
labs(
  title = "sigma_hat(k) vs rank (v11-style)",
  x = "Rank k (log scale)",
  y = "sigma_hat(k)"
) +
theme_bw()

```



Stability across time (quarterly summary)

```

xi_by_period <- xi_weekly %>%
  mutate(period = floor_date(week, params$bootstrap_period_unit)) %>%
  filter(k <= (K_xi_est - 1L))

rho_period_by_k <- xi_by_period %>%
  group_by(period, k) %>%
  summarise(rho_k = median(xi, na.rm = TRUE), .groups = "drop")

sigma_hat_period <- rho_period_by_k %>%
  inner_join(S_by_k %>% select(k, S_k), by = "k") %>%
  mutate(
    sigma2_hat = -2 * S_k * rho_k,
    sigma_hat = if_else(sigma2_hat > 0, sqrt(sigma2_hat), NA_real_)
  )

sigma_hat_period_summ <- sigma_hat_period %>%
  group_by(period) %>%
  summarise(

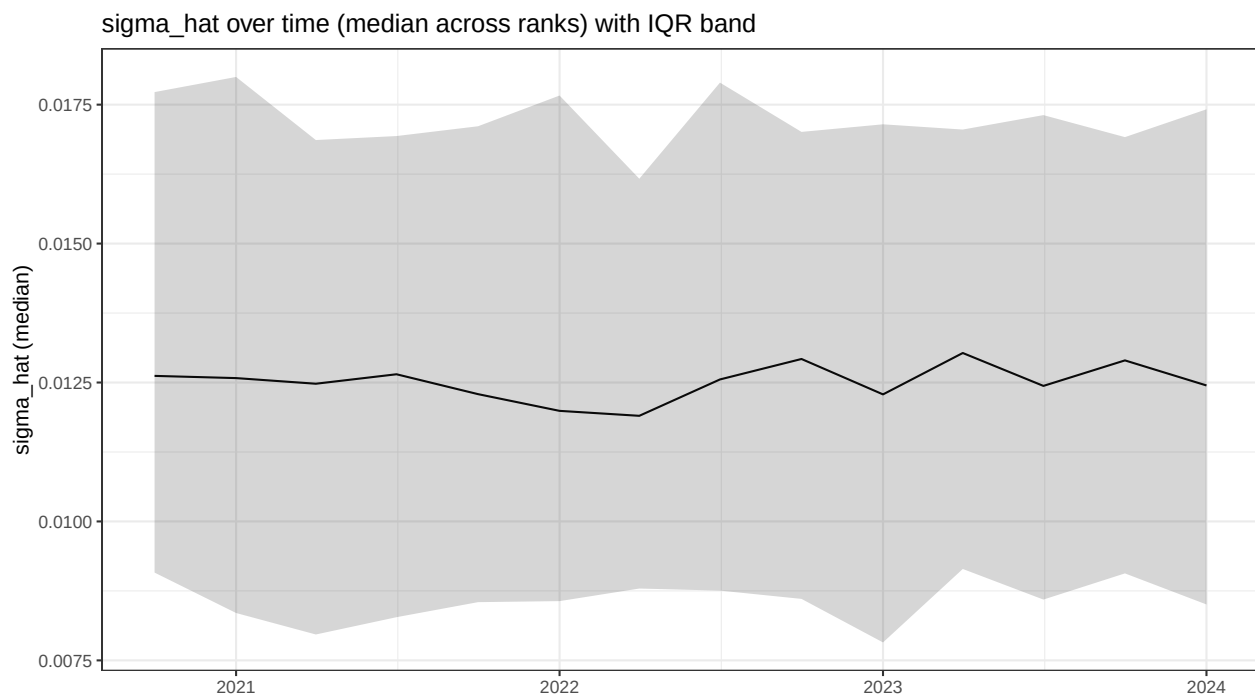
```

```

sigma_med = median(sigma_hat, na.rm = TRUE),
sigma_p25 = quantile(sigma_hat, 0.25, na.rm = TRUE),
sigma_p75 = quantile(sigma_hat, 0.75, na.rm = TRUE),
.groups = "drop"
)

ggplot(sigma_hat_period_summ, aes(x = period, y = sigma_med)) +
  geom_line() +
  geom_ribbon(aes(ymin = sigma_p25, ymax = sigma_p75), alpha = 0.2) +
  labs(
    title = "sigma_hat over time (median across ranks) with IQR band",
    x = NULL,
    y = "sigma_hat (median)"
  ) +
  theme_bw()

```



Bootstrap CI over time (v11-style)

```

bootstrap_sigma_period <- function(
  xi_df_period,
  S_by_k,
  B = 200,
  k_min = 10L,
  K_xi_est = 2000L
) {
  weeks <- unique(xi_df_period$week)
  boot_stats <- numeric(B)

  for (b in seq_len(B)) {
    w_samp <- sample(weeks, size = length(weeks), replace = TRUE)

```

```

rho_b <- xi_df_period %>%
  filter(week %in% w_samp, k <= (K_xi_est - 1L)) %>%
  group_by(k) %>%
  summarise(rho_k = median(xi, na.rm = TRUE), .groups = "drop")

sig_b <- rho_b %>%
  inner_join(S_by_k %>% select(k, S_k), by = "k") %>%
  mutate(
    sigma2_hat = -2 * S_k * rho_k,
    sigma_hat = if_else(sigma2_hat > 0, sqrt(sigma2_hat), NA_real_)
  ) %>%
  filter(k >= k_min, k <= (K_xi_est - 1L))

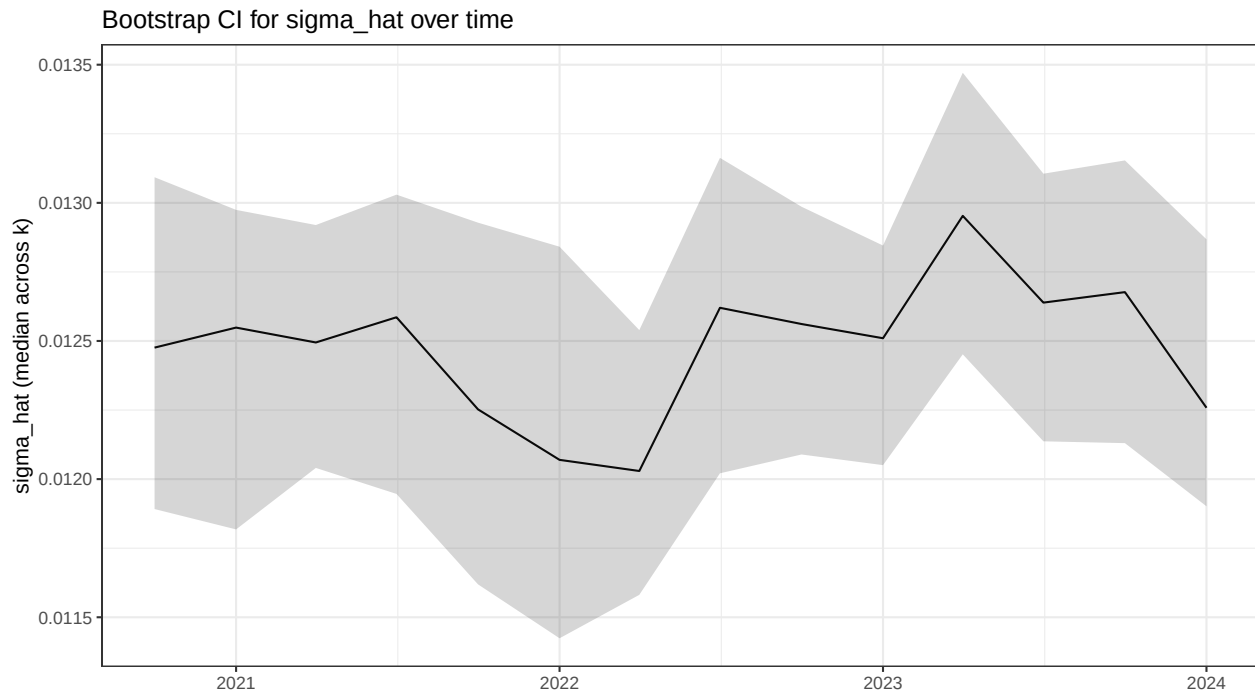
boot_stats[b] <- median(sig_b$sigma_hat, na.rm = TRUE)
}

tibble(
  sigma_hat_med = median(boot_stats, na.rm = TRUE),
  ci_lo = quantile(boot_stats, 0.025, na.rm = TRUE),
  ci_hi = quantile(boot_stats, 0.975, na.rm = TRUE)
)
}

sigma_ci_by_period <- xi_by_period %>%
  group_by(period) %>%
  group_modify(~ bootstrap_sigma_period(
    xi_df_period = .x,
    S_by_k = S_by_k,
    B = as.integer(bootstrap_B),
    k_min = as.integer(bootstrap_k_min),
    K_xi_est = K_xi_est
  )) %>%
  ungroup()

ggplot(sigma_ci_by_period, aes(x = period, y = sigma_hat_med)) +
  geom_line() +
  geom_ribbon(aes(ymin = ci_lo, ymax = ci_hi), alpha = 0.2) +
  labs(
    title = "Bootstrap CI for sigma_hat over time",
    x = NULL,
    y = "sigma_hat (median across k)"
  ) +
  theme_bw()

```



Stylized fact C (expanded): Turnover and rank mobility (new)

This section adds rank-mobility summaries that were not fully developed in the PDFs.

9.1 Top-K membership turnover (set overlap)

```
topK_overlap <- function(df, K, horizons = c(1L, 4L, 12L)) {
  K <- as.integer(K)

  df_topK <- df %>%
    filter(rank <= K) %>%
    select(week, endpoint_id) %>%
    distinct()

  week_vec <- sort(unique(df_topK$week))

  purrr::map_dfr(as.integer(horizons), function(h) {
    base_weeks <- week_vec[week_vec + lubridate::weeks(h) <= max(week_vec)]

    purrr::map_dfr(base_weeks, function(w) {
      w_tp <- w + lubridate::weeks(h)

      a <- df_topK %>% filter(week == w) %>% pull(endpoint_id)
      b <- df_topK %>% filter(week == w_tp) %>% pull(endpoint_id)

      # guard against weeks where the top-K list is shorter than K
      if (length(a) < K || length(b) < K) {
        return(tibble(K = K, horizon = h, week = w, overlap_frac = NA_real_))
      }

      tibble(
```

```

    K = K,
    horizon = h,
    week = w,
    overlap_frac = length(intersect(a, b)) / K
  )
})
}

turnover_tbl <- purrr::map_dfr(
  turnover_K_list,
  ~ topK_overlap(endpoint_weekly, K = .x, horizons = turnover_horizons)
)

ggplot(turnover_tbl, aes(x = week, y = overlap_frac, color = factor(horizon))) +
  geom_line(alpha = 0.6) +
  facet_wrap(~ K, scales = "free_y") +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    title = "Top-K overlap over time",
    subtitle = "Fraction of endpoints in top-K at week t that also appear in top-K at t+h",
    x = "Week",
    y = "Overlap fraction",
    color = "Horizon (weeks)"
  ) +
  theme_bw()

```



9.2 Rank-conditioned mobility: where does the endpoint at rank k go?

This extends the v12 micro-object by recording future rank as well as future share.

```

emp_micro_for_k <- function(endpoint_weekly, k, horizons = c(1L, 4L, 12L)) {
  base <- endpoint_weekly %>%
    filter(rank == k) %>%
    transmute(
      week_t = week,
      endpoint_id,
      share_t = share_topK
    )

  fut <- endpoint_weekly %>%
    transmute(
      week_tp = week,
      endpoint_id,
      rank_tp = rank,
      share_tp = share_topK
    )

  purrr::map_dfr(horizons, function(h) {
    base %>%
      mutate(week_tp = week_t + weeks(h), horizon = as.integer(h)) %>%
      left_join(fut, by = c("week_tp", "endpoint_id")) %>%
      mutate(
        present_tp = !is.na(share_tp),
        rank_change = if_else(present_tp, rank_tp - k, NA_integer_)
      )
  })
}

k_grid_mobility <- c(1L, 5L, 12L, 25L, 100L, 500L, 1000L)

emp_micro_mobility <- purrr::map_dfr(
  k_grid_mobility,
  ~ emp_micro_for_k(endpoint_weekly, k = .x, horizons = horizons_micro) %>%
  mutate(k = .x)
)

mobility_summ <- emp_micro_mobility %>%
  group_by(k, horizon) %>%
  summarise(
    retention = mean(present_tp),
    p_within_1 = mean(abs(rank_change) <= 1, na.rm = TRUE),
    p_within_5 = mean(abs(rank_change) <= 5, na.rm = TRUE),
    med_abs_rank_change = median(abs(rank_change), na.rm = TRUE),
    .groups = "drop"
  )

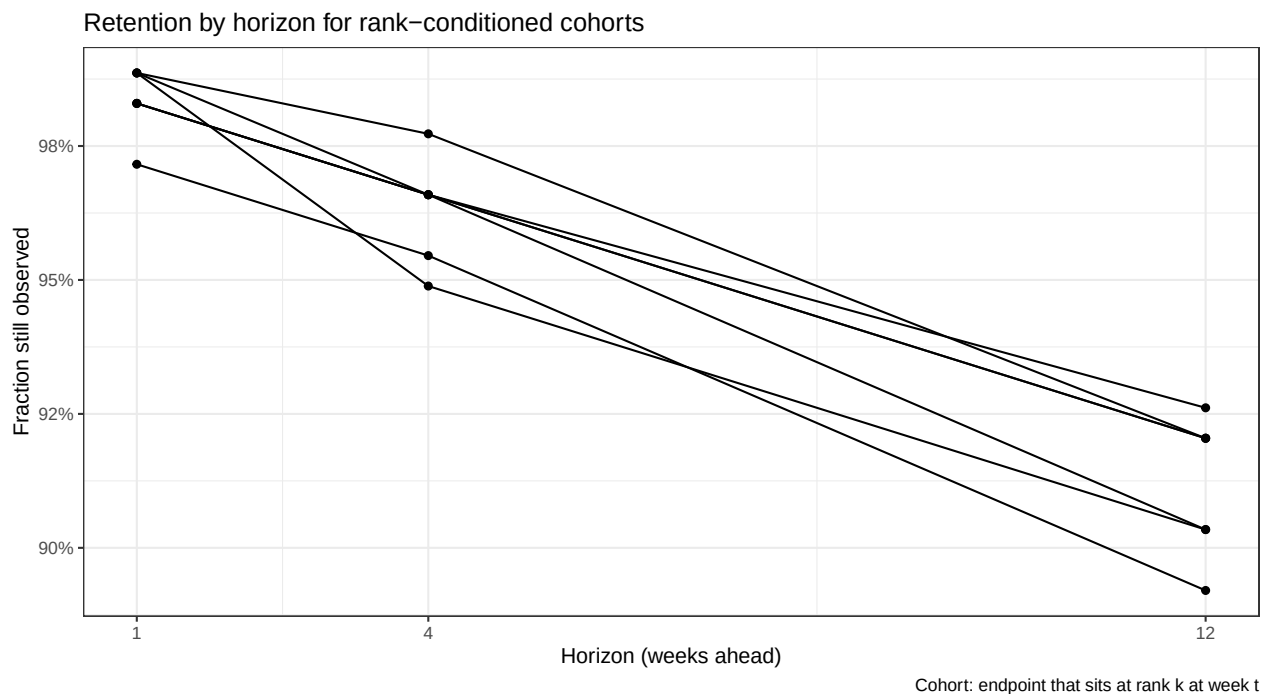
mobility_summ

## # A tibble: 21 x 6
##       k horizon retention p_within_1 p_within_5 med_abs_rank_change
##   <int>   <int>      <dbl>      <dbl>      <dbl>              <dbl>
## 1     1     1      0.989      0.672      0.856                1
## 2     1     4      0.966      0.447      0.735                2
## 3     1    12      0.920      0.346      0.623                3

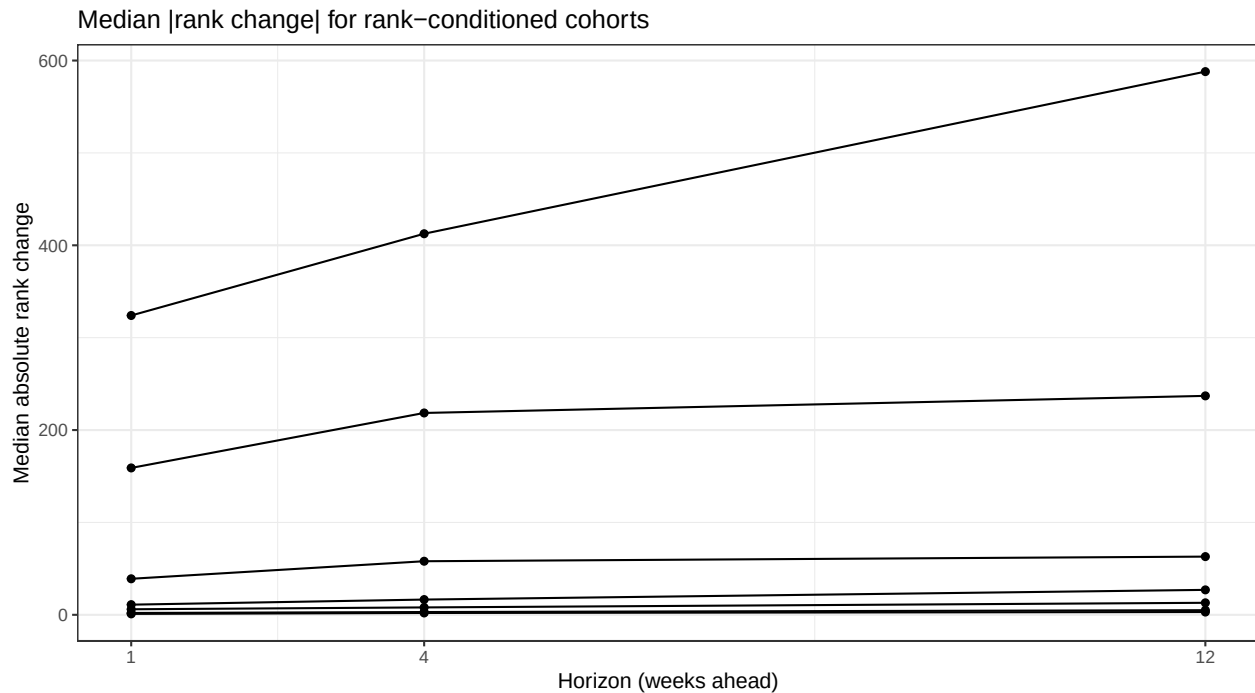
```

```
## 4      5      1      0.989      0.339      0.747      2
## 5      5      4      0.977      0.262      0.645      3
## 6      5     12      0.920      0.179      0.562      5
## 7     12      1      0.983      0.185      0.491      6
## 8     12      4      0.966      0.106      0.376      8
## 9     12     12      0.920      0.0617     0.259     13
## 10    25      1      0.983      0.0983     0.277     11
## # i 11 more rows
```

```
ggplot(mobility_summ, aes(x = horizon, y = retention, group = factor(k))) +
  geom_line() +
  geom_point(size = 1.5) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  scale_x_continuous(breaks = horizons_micro) +
  labs(
    title = "Retention by horizon for rank-conditioned cohorts",
    x = "Horizon (weeks ahead)",
    y = "Fraction still observed",
    caption = "Cohort: endpoint that sits at rank k at week t"
  ) +
  theme_bw()
```



```
ggplot(mobility_summ, aes(x = horizon, y = med_abs_rank_change, group = factor(k))) +
  geom_line() +
  geom_point(size = 1.5) +
  scale_x_continuous(breaks = horizons_micro) +
  labs(
    title = "Median |rank change| for rank-conditioned cohorts",
    x = "Horizon (weeks ahead)",
    y = "Median absolute rank change"
  ) +
  theme_bw()
```



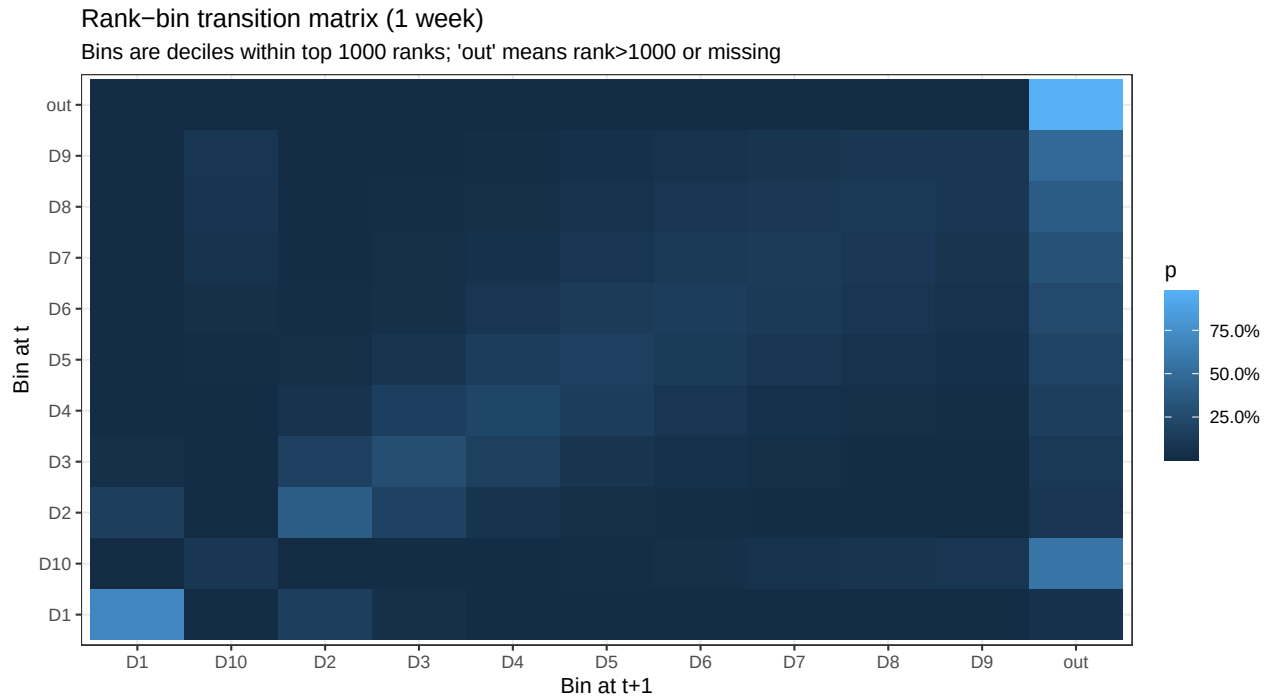
9.3 Rank transition matrix (deciles + out-of-topK)

```
make_rank_bin <- function(rank, K, n_bins = 10L) {
  case_when(
    is.na(rank) ~ "out",
    rank > K ~ "out",
    TRUE ~ paste0("D", pmin(n_bins, pmax(1L, ceiling(n_bins * rank / K))))
  )
}

rank_bins <- endpoint_weekly %>%
  select(week, endpoint_id, rank) %>%
  mutate(bin = make_rank_bin(rank, K = 1000L, n_bins = 10L))

# 1-week transition (calendar weeks; missing at t+1 treated as out)
trans_1w <- rank_bins %>%
  transmute(
    endpoint_id,
    week_t = week,
    bin_t = bin,
    week_tp = week + lubridate::weeks(1L)
  ) %>%
  left_join(
    rank_bins %>% transmute(endpoint_id, week_tp = week, bin_tp = bin),
    by = c("endpoint_id", "week_tp")
  ) %>%
  mutate(bin_tp = replace_na(bin_tp, "out")) %>%
  count(bin_t, bin_tp) %>%
  group_by(bin_t) %>%
  mutate(p = n / sum(n)) %>%
  ungroup()
```

```
ggplot(trans_1w, aes(x = bin_tp, y = bin_t, fill = p)) +
  geom_tile() +
  scale_fill_continuous(labels = percent_format(accuracy = 0.1)) +
  labs(
    title = "Rank-bin transition matrix (1 week)",
    subtitle = "Bins are deciles within top 1000 ranks; 'out' means rank>1000 or missing",
    x = "Bin at t+1",
    y = "Bin at t"
  ) +
  theme_bw()
```



Stylized fact E: Common shocks vs idiosyncratic movement (new)

A pragmatic diagnostic: PCA on rank-slot increments for the top `pca_K` ranks.

```
K_pca <- min(pca_K, K_cut)

if (is.na(K_pca) || K_pca < 2) {
  message("Skipping PCA: K_pca < 2 (K_pca=", K_pca, ").")
} else {
  rank_slot_wide <- rank_panel %>%
    filter(rank <= K_pca) %>%
    select(week, rank, dlogw) %>%
    pivot_wider(names_from = rank, values_from = dlogw)

  # keep complete weeks for PCA (top ranks usually complete)
  rank_slot_mat <- rank_slot_wide %>%
    drop_na() %>%
    arrange(week)

  X <- rank_slot_mat %>%
```

```

select(-week) %>%
as.matrix()

if (nrow(X) < 2 || ncol(X) < 2) {
  message(
    "Skipping PCA: need at least 2 complete weeks and 2 rank columns; got ",
    nrow(X), "x", ncol(X), "."
  )
} else {
  pca_fit <- prcomp(X, center = TRUE, scale. = TRUE)

  var_explained <- (pca_fit$sdev^2) / sum(pca_fit$sdev^2)

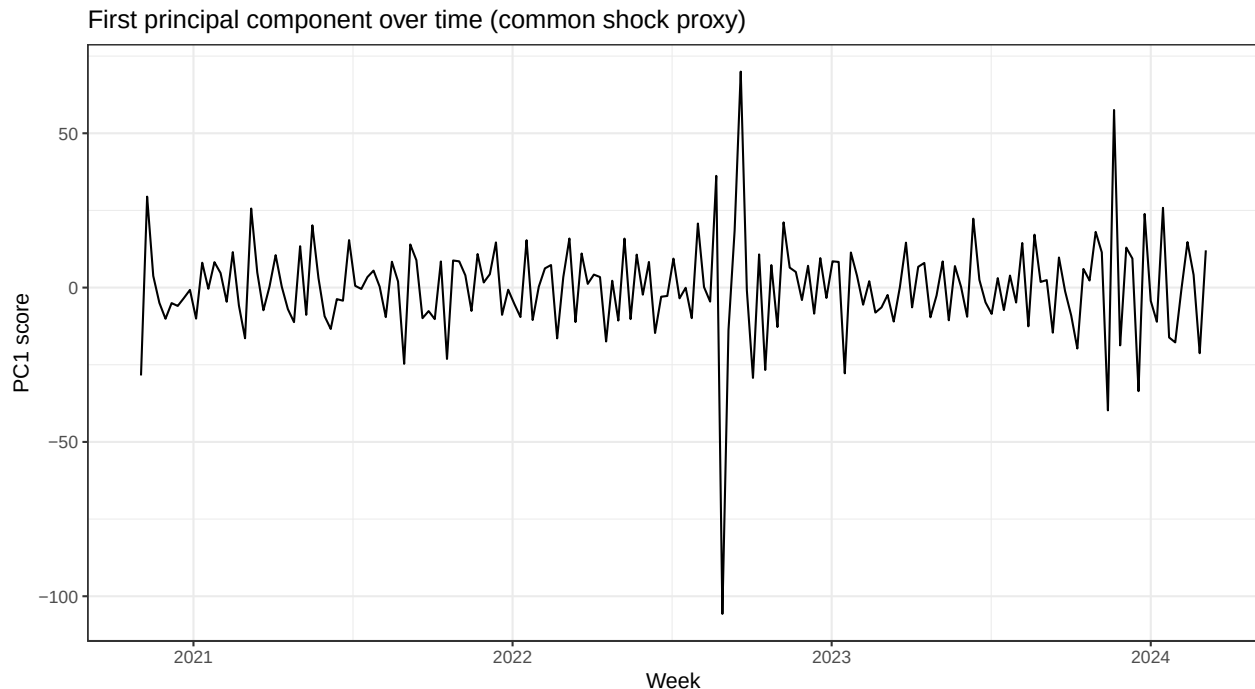
  tibble(
    component = seq_along(var_explained),
    var_explained = var_explained
  ) %>%
    slice(1:10)

  ggplot(
    tibble(component = seq_along(var_explained), var_explained = var_explained) %>% slice(1:20),
    aes(component, var_explained)
  ) +
    geom_line() +
    geom_point(size = 1.5) +
    scale_y_continuous(labels = percent_format(accuracy = 0.1)) +
    labs(
      title = "PCA on rank-slot increments: variance explained",
      subtitle = paste0("Top ", K_pca, " ranks; complete weeks only"),
      x = "Principal component",
      y = "Share of variance explained"
    ) +
    theme_bw()

  # first PC time series (common shock proxy)
  pc1 <- pca_fit$x[, 1]
  pc1_df <- tibble(week = rank_slot_mat$week, pc1 = pc1)

  ggplot(pc1_df, aes(week, pc1)) +
    geom_line() +
    labs(
      title = "First principal component over time (common shock proxy)",
      x = "Week",
      y = "PC1 score"
    ) +
    theme_bw()
}
}

```



Heavy-tail / jump diagnostics for increments (new)

This section checks whether rank-slot increments have tails that are substantially heavier than Gaussian.

```
# Build standardized increments by bucket
inc_by_bucket <- rank_panel %>%
  filter(rank <= K_cut) %>%
  mutate(bucket = assign_bucket(rank, bucket_def)) %>%
  filter(!is.na(bucket)) %>%
  group_by(bucket) %>%
  mutate(
    z = (dlogw - mean(dlogw, na.rm = TRUE)) / sd(dlogw, na.rm = TRUE)
  ) %>%
  ungroup()

tail_summ <- inc_by_bucket %>%
  group_by(bucket) %>%
  summarise(
    n = n(),
    p_gt_5sd = mean(abs(z) >= 5, na.rm = TRUE),
    p_gt_3sd = mean(abs(z) >= 3, na.rm = TRUE),
    q_0.001 = quantile(z, 0.001, na.rm = TRUE),
    q_0.999 = quantile(z, 0.999, na.rm = TRUE),
    .groups = "drop"
  )

tail_summ
```

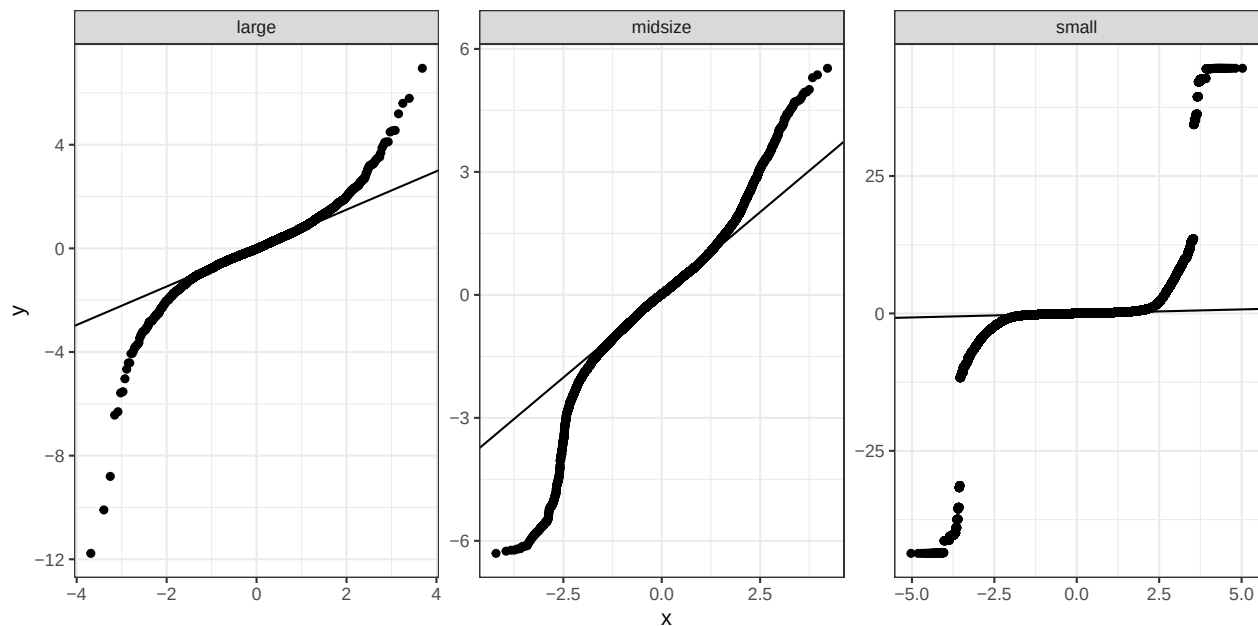
```
## # A tibble: 3 x 6
##   bucket      n p_gt_5sd p_gt_3sd q_0.001 q_0.999
##   <fct>    <int>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 large    4375  0.00274  0.0146  -6.03    4.55
```

```
## 2 midsize 39375 0.0032 0.0145 -5.68 4.17
## 3 small 2021072 0.00393 0.00854 -6.24 7.76
```

```
ggplot(inc_by_bucket, aes(sample = z)) +
  stat_qq() +
  stat_qq_line() +
  facet_wrap(~ bucket, scales = "free") +
  labs(
    title = "QQ plot of standardized increments by bucket",
    subtitle = "Visual check for heavy tails vs Gaussian"
  ) +
  theme_bw()
```

QQ plot of standardized increments by bucket

Visual check for heavy tails vs Gaussian



Simulation framework (v8–v11) + unified fit metrics

12.1 Empirical entrant pool for entry/exit

```
entrant_pool <- endpoint_weekly %>%
  filter(rank > K_cut, rank <= min(K_cut + K_tail_buffer, max_rank_seen)) %>%
  transmute(share = share_global) %>%
  filter(is.finite(share), share > 0)

stopifnot(nrow(entrant_pool) > 100)

entrant_sampler <- function(n) {
  sample(entrant_pool$share, size = n, replace = TRUE)
}

# baseline seed curve w0 up to K_max
mean_share <- endpoint_weekly %>%
  group_by(rank) %>%
  summarise(w_bar = mean(share_global, na.rm = TRUE), .groups = "drop") %>%
```

```

arrange(rank)

w0_raw <- mean_share$w_bar[seq_len(min(K_max, nrow(mean_share)))]
w0_raw <- w0_raw / sum(w0_raw)

# extend once to full universe length K_max
if (length(w0_raw) < K_max) {
  w0_ext <- c(w0_raw, entrant_sampler(K_max - length(w0_raw)))
  w0_ext <- w0_ext / sum(w0_ext)
} else {
  w0_ext <- w0_raw[1:K_max]
  w0_ext <- w0_ext / sum(w0_ext)
}

```

12.2 Simulator (Gaussian increments with smoothed moments; optional Xi output)

This merges v9's simulator with v11's Xi storage.

```

simulate_rank_paths <- function(
  w0, K_cut, K_max, T = 52, n_paths = 200,
  mu = 0, sigma = 1, entry_frac = 0.10,
  mean_vec, sd_vec,
  horizons = c(4L, 8L),
  K_xi = NULL
) {
  stopifnot(length(mean_vec) == K_max, length(sd_vec) == K_max)

  top_n_sorted <- function(w, n) w[order(w, decreasing = TRUE)][seq_len(n)]

  # sample one Gaussian increment per slot
  sample_increments_gaussian <- function(mu = 0, sigma = 1) {
    rnorm(K_max, mean = mu + mean_vec, sd = sigma * sd_vec)
  }

  horizons <- as.integer(horizons)
  stopifnot(all(horizons >= 1L), all(horizons <= T))

  if (!is.null(K_xi)) {
    K_xi <- as.integer(K_xi)
    stopifnot(K_xi >= 2L, K_xi <= K_cut)
  }

  snapshots <- vector("list", n_paths)
  growth_collect <- setNames(vector("list", length(horizons)), paste0("h", horizons))
  xi_collect <- vector("list", n_paths)

  for (p in seq_len(n_paths)) {
    # initialize to full universe length K_max
    if (length(w0) < K_max) {
      w <- c(w0, entrant_sampler(K_max - length(w0)))
      w <- w / sum(w)
    } else {
      w <- w0[1:K_max]
    }
  }

```

```

  w <- w / sum(w)
}

# baseline top-K for durable change (within-top-K normalization)
w_baseK <- top_n_sorted(w, K_cut)
w_baseK <- w_baseK / sum(w_baseK)

snaps_p <- vector("list", T)
xi_p <- vector("list", T)

for (t in seq_len(T)) {

  dlogw <- sample_increments_gaussian(mu = mu, sigma = sigma)
  w <- w * exp(dlogw)
  w <- pmax(w, 1e-18)
  w <- w / sum(w)

  # entry/exit: keep top-K_cut, refill tail to K_max
  w_top <- top_n_sorted(w, K_cut) # global shares of current top-K
  n_tail <- K_max - K_cut

  if (n_tail > 0) {
    n_add <- max(1L, floor(n_tail * entry_frac))
    w <- c(w_top, entrant_sampler(n_add), entrant_sampler(n_tail - n_add))
  } else {
    w <- w_top
  }

  w <- w / sum(w)

  # snapshot (top-K, within-top-K normalization)
  wK <- top_n_sorted(w, K_cut)
  wK <- wK / sum(wK)

  snaps_p[[t]] <- tibble(
    path = p,
    t = t,
    rank = seq_len(K_cut),
    share = wK
  )

  # Xi (global shares; scale-invariant, but use current top-K global vector)
  if (!is.null(K_xi)) {
    w_top_now <- top_n_sorted(w, K_cut) %>% pmax(1e-18)
    xi_vals <- log(w_top_now[seq_len(K_xi - 1L)] / w_top_now[seq(2L, K_xi)])
    xi_p[[t]] <- tibble(
      path = p,
      t = t,
      k = seq_len(K_xi - 1L),
      xi = xi_vals
    )
  }
}

```

```

# durable change at selected horizons relative to baseline (top-K, within-top-K)
if (t %in% horizons) {
  lg <- log(wK / w_baseK)
  lbl <- paste0("h", t)
  growth_collect[[lbl]] <- rbind(
    growth_collect[[lbl]],
    cbind(
      path = p,
      rank = seq_len(K_cut),
      horizon = t,
      log_growth = lg
    )
  )
}
}

snapshots[[p]] <- bind_rows(snaps_p)
if (!is.null(K_xi)) xi_collect[[p]] <- bind_rows(xi_p)
}

growth_df <- bind_rows(lapply(growth_collect, as_tibble))

out <- list(
  snapshots = bind_rows(snapshots),
  growth = growth_df
)

if (!is.null(K_xi)) out$xi <- bind_rows(xi_collect)
out
}

```

12.3 Fit metrics (CDC, durable change, Xi)

```

cdc_rmse <- function(emp, sim) {
  df <- emp %>%
    inner_join(sim, by = "rank", suffix = c("_emp", "_sim"))
  sqrt(mean((log(df$w_bar_emp) - log(df$w_bar_sim))^2, na.rm = TRUE))
}

durable_rmse <- function(emp_targets, sim_growth, bucket_def) {
  sim_targets <- sim_growth %>%
    mutate(bucket = assign_bucket(rank, bucket_def)) %>%
    filter(!is.na(bucket)) %>%
    group_by(bucket, horizon) %>%
    summarise(med_abs_sim = median(abs(log_growth), na.rm = TRUE), .groups = "drop")

  emp_targets %>%
    inner_join(sim_targets, by = c("bucket", "horizon")) %>%
    summarise(rmse = sqrt(mean((med_abs_emp - med_abs_sim)^2, na.rm = TRUE))) %>%
    pull(rmse)
}

xi_rmse <- function(emp_rho_by_k, sim_xi_df) {

```

```

sim_rho <- sim_xi_df %>%
  group_by(k) %>%
  summarise(rho_k = median(xi, na.rm = TRUE), .groups = "drop")
df <- emp_rho_by_k %>%
  inner_join(sim_rho, by = "k", suffix = c("_emp", "_sim"))
sqrt(mean((df$rho_k_emp - df$rho_k_sim)^2, na.rm = TRUE))
}

```

Baseline simulation run + comparison (CDC, durable, Xi)

```

sm_fixed <- build_gaussian_params_smoothed(h = as.integer(Smoothing_h))
mean_vec_s <- sm_fixed$mean_dlogw_s
sd_vec_s <- sm_fixed$sd_dlogw_s

sim_out <- simulate_rank_paths(
  w0 = w0_ext,
  K_cut = K_cut,
  K_max = K_max,
  T = sim_T_weeks,
  n_paths = sim_n_paths,
  mu = sim_mu,
  sigma = sim_sigma,
  entry_frac = sim_entry_frac,
  mean_vec = mean_vec_s,
  sd_vec = sd_vec_s,
  horizons = horizons_durable,
  K_xi = sim_K_xi
)

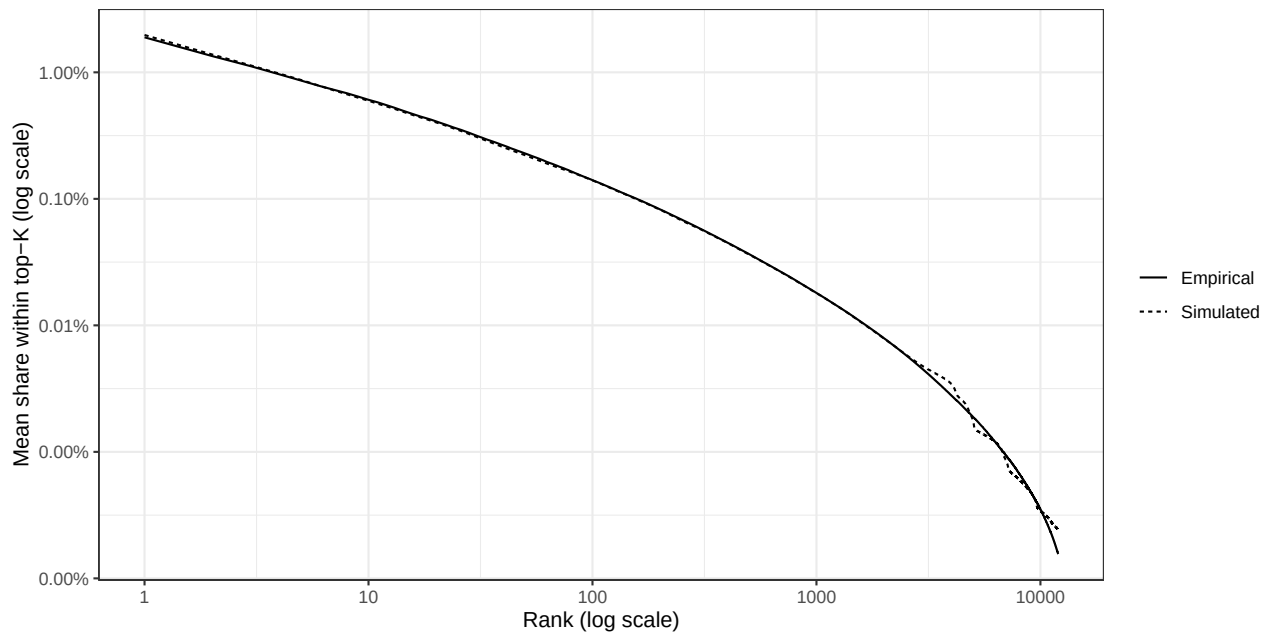
# CDC comparison
sim_cdc <- sim_out$snapshots %>%
  group_by(rank) %>%
  summarise(w_bar = mean(share, na.rm = TRUE), .groups = "drop") %>%
  arrange(rank)

bind_rows(
  emp_cdc %>% mutate(source = "Empirical"),
  sim_cdc %>% mutate(source = "Simulated")
) %>%
  ggplot(aes(rank, w_bar, linetype = source)) +
  geom_line() +
  scale_x_log10() +
  scale_y_log10(labels = percent_format(accuracy = 0.01)) +
  labs(
    title = "CDC: Empirical vs Simulated",
    subtitle = paste0("mu=", sim_mu, ", sigma=", sim_sigma,
                      ", h=", Smoothing_h),
    x = "Rank (log scale)",
    y = "Mean share within top-K (log scale)",
    linetype = NULL
  ) +
  theme_bw()

```

CDC: Empirical vs Simulated

$\mu=0$, $\sigma=0.16$, $h=5$



```
# Scoreboard
score_tbl <- tibble(
  rmse_cdc = cdc_rmse(emp_cdc, sim_cdc),
  rmse_durable = durable_rmse(emp_targets, sim_out$growth, bucket_def),
  rmse_xi = xi_rmse(rho_by_k_med %>% filter(k <= sim_K_xi - 1L),
                    sim_out$xi %>% filter(k <= sim_K_xi - 1L))
)

score_tbl
```

```
## # A tibble: 1 x 3
##   rmse_cdc rmse_durable rmse_xi
##   <dbl>      <dbl>    <dbl>
## 1    0.126      0.0552 0.00790
```

Micro-level movement validation (v12) + rank-slot comparator

14.1 Empirical micro object (identity-based, v12)

```
emp_micro_k12 <- emp_micro_for_k(
  endpoint_weekly = endpoint_weekly %>% filter(rank <= K_cut),
  k = 12L,
  horizons = horizons_micro
)

emp_summary_k12 <- emp_micro_k12 %>%
  group_by(horizon) %>%
  summarise(
    n_total = n(),
    retention = mean(present_tp),
    n_present = sum(present_tp),
    med = median(share_tp[present_tp], na.rm = TRUE),
```

```

p10 = quantile(share_tp[present_tp], 0.10, na.rm = TRUE),
p90 = quantile(share_tp[present_tp], 0.90, na.rm = TRUE),
.groups = "drop"
) %>%
mutate(source = "empirical")

```

14.2 Simulation micro object (rank-slot, v12 style)

```

sim_micro_for_k <- function(sim_snapshots, k, horizons = c(1L, 4L, 12L)) {

  base <- sim_snapshots %>%
    filter(rank == k) %>%
    transmute(path, t, share_t = share)

  fut <- sim_snapshots %>%
    filter(rank == k) %>%
    transmute(path, t_tp = t, share_tp = share)

  purrr::map_dfr(horizons, function(h) {
    base %>%
      mutate(t_tp = t + as.integer(h), horizon = as.integer(h)) %>%
      left_join(fut, by = c("path", "t_tp")) %>%
      filter(!is.na(share_tp))
  })
}

sim_micro_k12 <- sim_micro_for_k(
  sim_snapshots = sim_out$snapshots,
  k = 12L,
  horizons = horizons_micro
)

sim_summary_k12 <- sim_micro_k12 %>%
  group_by(horizon) %>%
  summarise(
    n_total = n(),
    retention = 1.0,
    n_present = n(),
    med = median(share_tp, na.rm = TRUE),
    p10 = quantile(share_tp, 0.10, na.rm = TRUE),
    p90 = quantile(share_tp, 0.90, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(source = "simulation")

micro_summary_k12 <- bind_rows(emp_summary_k12, sim_summary_k12)

micro_summary_k12

```

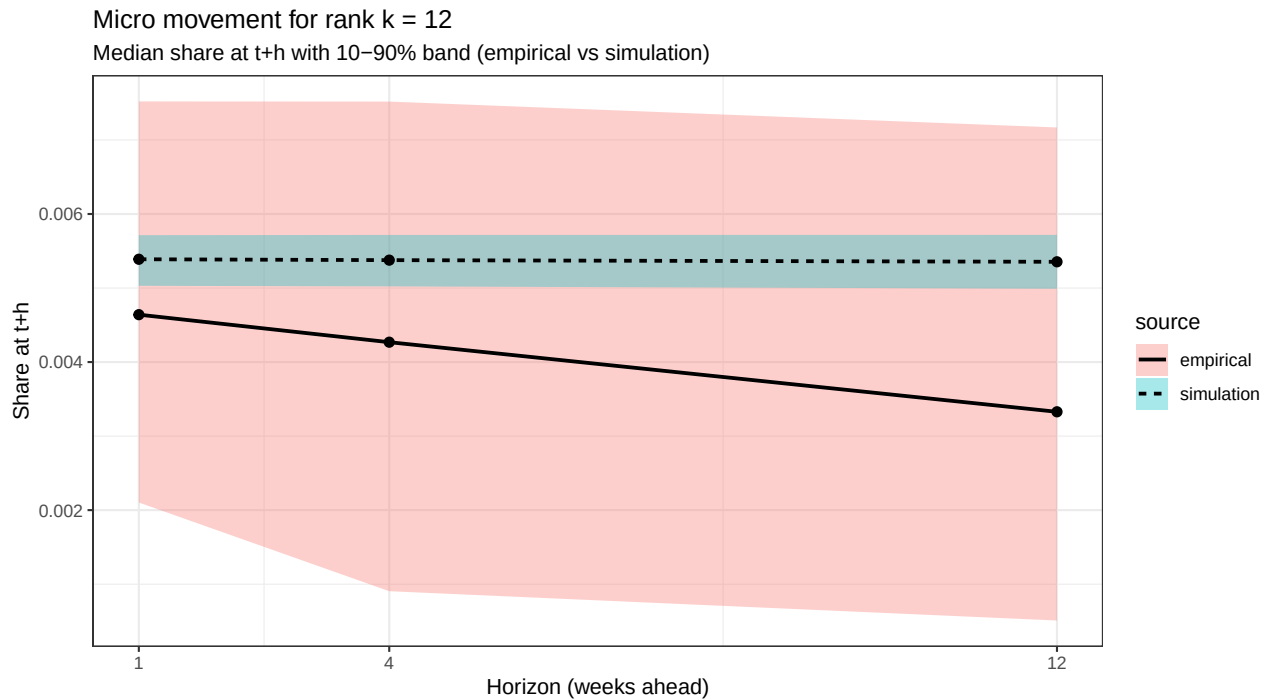
```

## # A tibble: 6 x 8
##   horizon n_total retention n_present    med    p10    p90 source
##   <int>   <int>   <dbl>   <int>  <dbl>  <dbl>  <dbl> <chr>
## 1       1     176    0.983     173 0.00464 0.00210 0.00752 empirical

```

```
## 2      4      176      0.966      170 0.00427 0.000905 0.00752 empirical
## 3     12      176      0.920      162 0.00333 0.000509 0.00717 empirical
## 4      1    10200      1      10200 0.00539 0.00503 0.00571 simulation
## 5      4     9600      1      9600 0.00538 0.00502 0.00572 simulation
## 6     12     8000      1      8000 0.00535 0.00499 0.00572 simulation
```

```
ggplot(micro_summary_k12, aes(x = horizon, y = med, group = source, linetype = source)) +
  geom_ribbon(
    aes(ymin = p10, ymax = p90, fill = source),
    alpha = 0.35,
    color = NA
  ) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = horizons_micro) +
  labs(
    title = "Micro movement for rank k = 12",
    subtitle = "Median share at t+h with 10-90% band (empirical vs simulation)",
    x = "Horizon (weeks ahead)",
    y = "Share at t+h"
  ) +
  theme_bw()
```



14.3 Empirical rank-slot micro comparator (new)

This uses the *rank-slot* share at t+h (not tracking endpoint identity), to make an apples-to-apples comparison with the simulation object.

```
emp_rankslot_micro_for_k <- function(endpoint_weekly, k, horizons = c(1L, 4L, 12L)) {
  base <- endpoint_weekly %>%
    filter(rank == k) %>%
    transmute(week_t = week, share_t = share_topK)
```

```

fut <- endpoint_weekly %>%
  filter(rank == k) %>%
  transmute(week_tp = week, share_tp = share_topK)

purrr::map_dfr(horizons, function(h) {
  base %>%
    mutate(week_tp = week_t + weeks(as.integer(h)), horizon = as.integer(h)) %>%
    left_join(fut, by = "week_tp") %>%
    filter(!is.na(share_tp))
})
}

emp_rs_k12 <- emp_rankslot_micro_for_k(
  endpoint_weekly = endpoint_weekly %>% filter(rank <= K_cut),
  k = 12L,
  horizons = horizons_micro
)

emp_rs_summary_k12 <- emp_rs_k12 %>%
  group_by(horizon) %>%
  summarise(
    n_total = n(),
    med = median(share_tp, na.rm = TRUE),
    p10 = quantile(share_tp, 0.10, na.rm = TRUE),
    p90 = quantile(share_tp, 0.90, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(source = "emp_rankslot")

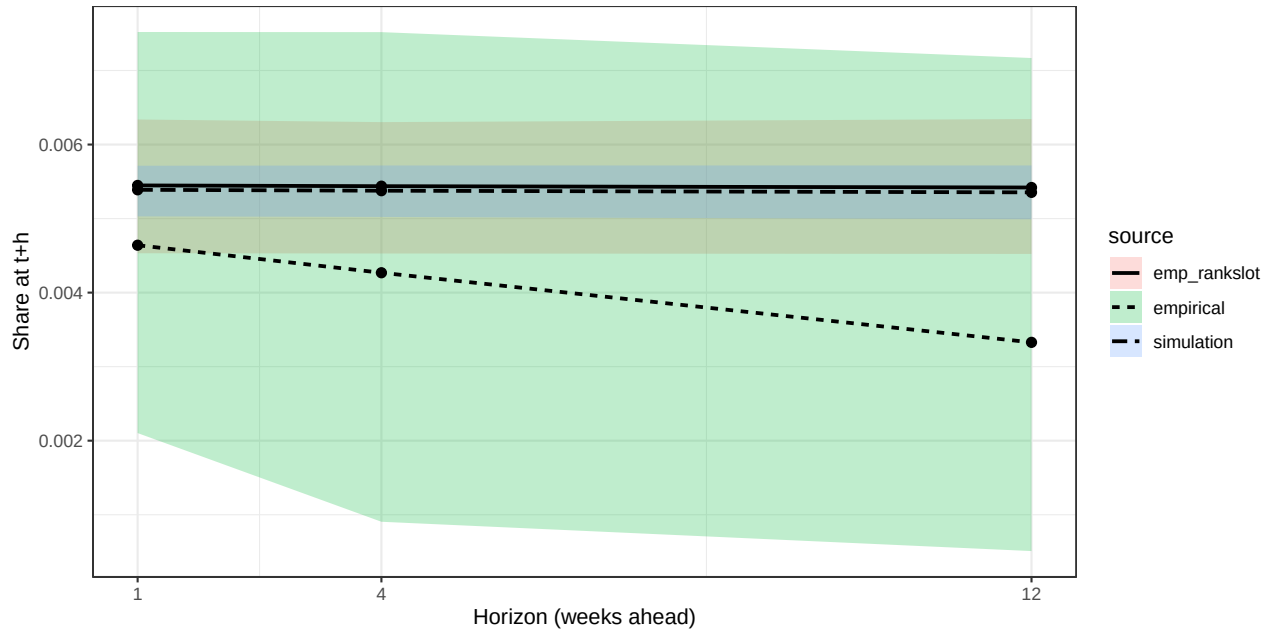
micro_summary_k12_3way <- bind_rows(
  emp_summary_k12 %>% select(horizon, med, p10, p90, source),
  emp_rs_summary_k12,
  sim_summary_k12 %>% select(horizon, med, p10, p90, source)
)

ggplot(micro_summary_k12_3way, aes(x = horizon, y = med, group = source, linetype = source)) +
  geom_ribbon(
    aes(ymin = p10, ymax = p90, fill = source),
    alpha = 0.25,
    color = NA
  ) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = horizons_micro) +
  labs(
    title = "Micro movement (k=12): identity vs rank-slot vs simulation",
    subtitle = "Empirical identity-tracking differs conceptually from rank-slot tracking",
    x = "Horizon (weeks ahead)",
    y = "Share at t+h"
  ) +
  theme_bw()

```

Micro movement (k=12): identity vs rank-slot vs simulation

Empirical identity-tracking differs conceptually from rank-slot tracking



Micro fit across ranks (v12-style)

This reproduces the v12 idea of scoring micro-movement fit across a grid of ranks (here: 10, 20, ..., 200). It compares empirical **identity-based** future shares (conditioned on presence at $t+h$) to the simulated **rank-slot** future shares.

```
k_grid_micro <- seq(10L, 200L, by = 10L)

micro_fit_tbl <- purrr::map_dfr(k_grid_micro, function(k) {

  emp <- emp_micro_for_k(
    endpoint_weekly = endpoint_weekly %>% filter(rank <= K_cut),
    k = k,
    horizons = horizons_micro
  ) %>%
  group_by(horizon) %>%
  summarise(
    emp_retention = mean(present_tp),
    emp_med = median(share_tp[present_tp], na.rm = TRUE),
    .groups = "drop"
  )

  sim <- sim_micro_for_k(
    sim_snapshots = sim_out$snapshots,
    k = k,
    horizons = horizons_micro
  ) %>%
  group_by(horizon) %>%
  summarise(
    sim_med = median(share_tp, na.rm = TRUE),
    .groups = "drop"
  )
})
```

```

emp %>%
  inner_join(sim, by = "horizon") %>%
  mutate(
    k = k,
    sq_err = (emp_med - sim_med)^2
  )
})

micro_rmse_by_k <- micro_fit_tbl %>%
  group_by(k) %>%
  summarise(
    rmse = sqrt(mean(sq_err, na.rm = TRUE)),
    avg_retention = mean(emp_retention, na.rm = TRUE),
    .groups = "drop"
  )

micro_rmse_by_k

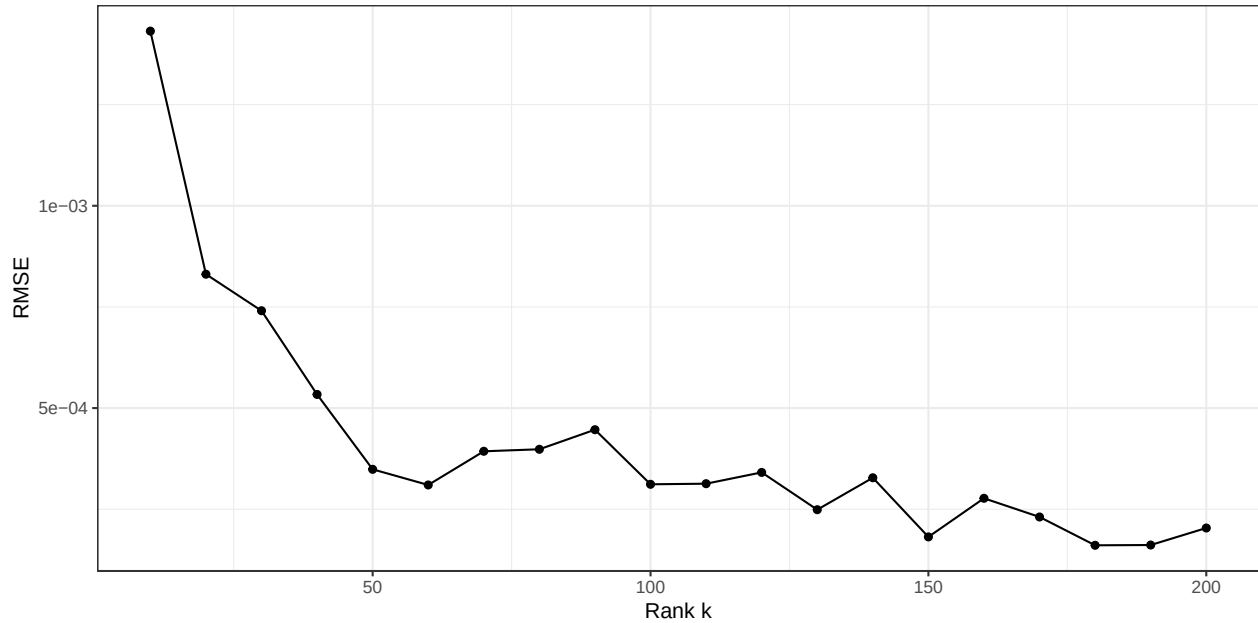
## # A tibble: 20 x 3
##       k      rmse avg_retention
##   <int>   <dbl>         <dbl>
## 1     10 0.00143         0.962
## 2     20 0.000831        0.958
## 3     30 0.000741        0.958
## 4     40 0.000534        0.953
## 5     50 0.000349        0.953
## 6     60 0.000310        0.960
## 7     70 0.000393        0.951
## 8     80 0.000398        0.956
## 9     90 0.000447        0.943
## 10    100 0.000312        0.947
## 11    110 0.000313        0.953
## 12    120 0.000341        0.956
## 13    130 0.000249        0.941
## 14    140 0.000328        0.953
## 15    150 0.000182        0.953
## 16    160 0.000277        0.941
## 17    170 0.000231        0.949
## 18    180 0.000161        0.955
## 19    190 0.000162        0.958
## 20    200 0.000204        0.955

ggplot(micro_rmse_by_k, aes(k, rmse)) +
  geom_line() +
  geom_point(size = 1.5) +
  labs(
    title = "Micro-movement RMSE by rank (k)",
    subtitle = "RMSE over horizons between empirical identity-based median(share_tp) and simulated median",
    x = "Rank k",
    y = "RMSE"
  ) +
  theme_bw()

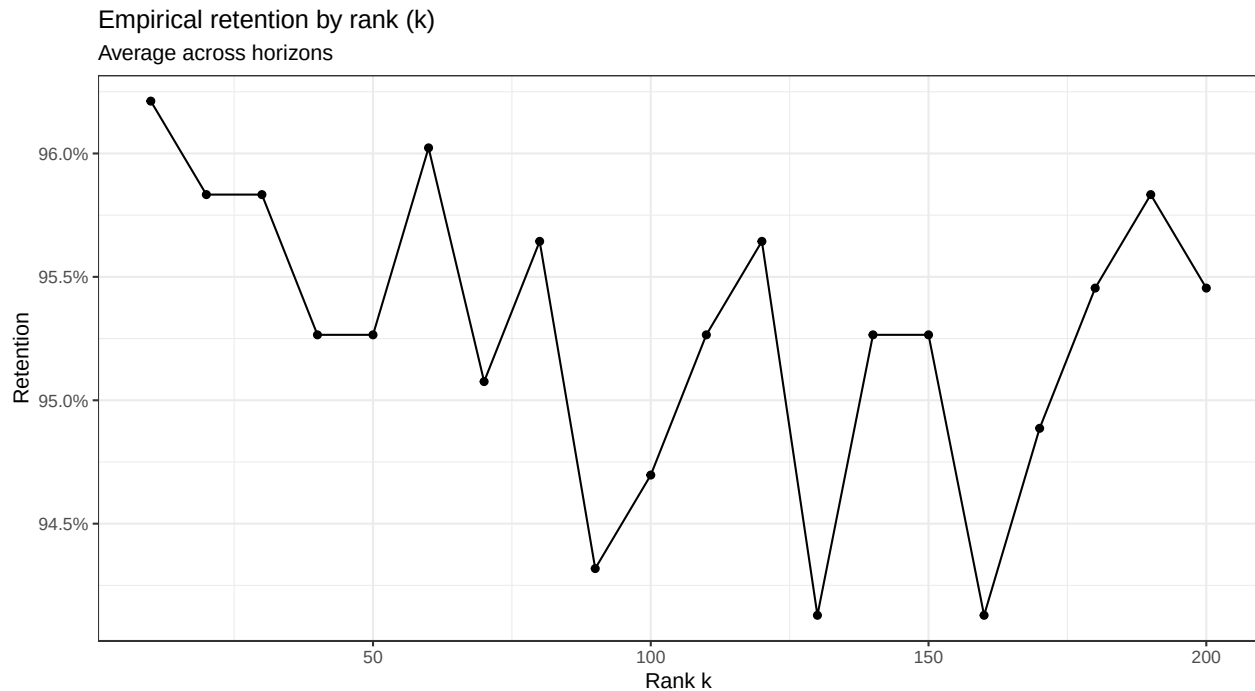
```

Micro-movement RMSE by rank (k)

RMSE over horizons between empirical identity-based median(share_tp) and simulated median(share_tp)



```
ggplot(micro_rmse_by_k, aes(k, avg_retention)) +  
  geom_line() +  
  geom_point(size = 1.5) +  
  scale_y_continuous(labels = percent_format(accuracy = 0.1)) +  
  labs(  
    title = "Empirical retention by rank (k)",  
    subtitle = "Average across horizons",  
    x = "Rank k",  
    y = "Retention"  
  ) +  
  theme_bw()
```



Sigma calibration loop (v10) (optional but included)

This reproduces the v10 approach: scan `sigma` and pick the value that minimizes durable mismatch, with a small CDC penalty.

```
calibrate_sigma <- function(sigma, w0_ext, mean_vec_s, sd_vec_s) {
  sim_res <- simulate_rank_paths(
    w0 = w0_ext,
    K_cut = K_cut,
    K_max = K_max,
    T = sim_T_weeks,
    n_paths = sim_n_paths,
    mu = sim_mu,
    sigma = sigma,
    entry_frac = sim_entry_frac,
    mean_vec = mean_vec_s,
    sd_vec = sd_vec_s,
    horizons = horizons_durable,
    K_xi = NULL
  )

  sim_cdc_i <- sim_res$snapshots %>%
    group_by(rank) %>%
    summarise(w_bar = mean(share, na.rm = TRUE), .groups = "drop")

  rmse_dur <- durable_rmse(emp_targets, sim_res$growth, bucket_def)
  rmse_cdc <- cdc_rmse(emp_cdc, sim_cdc_i)

  tibble(
    sigma = sigma,
    rmse_dur = rmse_dur,
    rmse_cdc = rmse_cdc,
  )
}
```

```

    score = rmse_dur + 0.25 * rmse_cdc
  )
}

sigma_grid <- seq(0.15, 0.40, by = 0.01)

sigma_results <- purrr::map_dfr(
  sigma_grid,
  calibrate_sigma,
  w0_ext = w0_ext,
  mean_vec_s = mean_vec_s,
  sd_vec_s = sd_vec_s
) %>%
  arrange(score)

best_sigma <- sigma_results %>% slice(1)
best_sigma

```

```

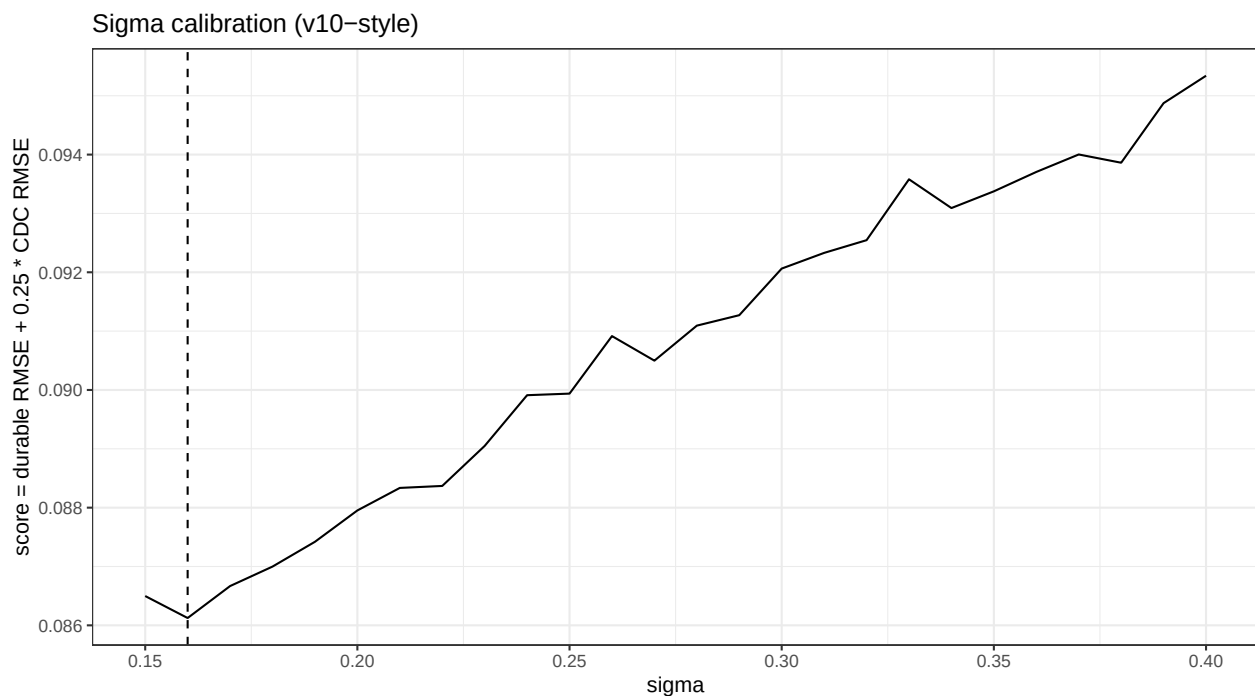
## # A tibble: 1 x 4
##   sigma rmse_dur rmse_cdc score
##   <dbl>   <dbl>   <dbl> <dbl>
## 1  0.16  0.0546   0.126 0.0861

```

```

ggplot(sigma_results, aes(x = sigma, y = score)) +
  geom_line() +
  geom_vline(xintercept = best_sigma$sigma, linetype = "dashed") +
  labs(
    title = "Sigma calibration (v10-style)",
    x = "sigma",
    y = "score = durable RMSE + 0.25 * CDC RMSE"
  ) +
  theme_bw()

```



Sigma calibration by horizon (v10-style) (optional)

This is the v10 extension: pick a separate `sigma` for each horizon by minimizing the horizon-specific durable-change mismatch.

```
if (isTRUE(params$run_grid_search)) {

  sigma_by_horizon <- function(sigma, w0_ext, mean_vec_s, sd_vec_s, bucket_def) {

    sim_res <- simulate_rank_paths(
      w0 = w0_ext,
      K_cut = K_cut,
      K_max = K_max,
      T = sim_T_weeks,
      n_paths = sim_n_paths,
      mu = sim_mu,
      sigma = sigma,
      entry_frac = sim_entry_frac,
      mean_vec = mean_vec_s,
      sd_vec = sd_vec_s,
      horizons = horizons_durable,
      K_xi = NULL
    )

    sim_targets <- sim_res$growth %>%
      mutate(bucket = assign_bucket(rank, bucket_def)) %>%
      filter(!is.na(bucket)) %>%
      group_by(bucket, horizon) %>%
      summarise(med_abs_sim = median(abs(log_growth), na.rm = TRUE), .groups = "drop")

    emp_targets %>%
      inner_join(sim_targets, by = c("bucket", "horizon")) %>%
      group_by(horizon) %>%
      summarise(
        sigma = sigma,
        rmse_h = sqrt(mean((med_abs_emp - med_abs_sim)^2, na.rm = TRUE)),
        .groups = "drop"
      )
  }

  sigma_h_tbl <- purrr::map_dfr(
    sigma_grid,
    sigma_by_horizon,
    w0_ext = w0_ext,
    mean_vec_s = mean_vec_s,
    sd_vec_s = sd_vec_s,
    bucket_def = bucket_def
  )

  best_sigma_by_horizon <- sigma_h_tbl %>%
    group_by(horizon) %>%
    arrange(rmse_h, .by_group = TRUE) %>%
    slice(1) %>%
    ungroup()
}
```

```

best_sigma_by_horizon

ggplot(sigma_h_tbl, aes(x = sigma, y = rmse_h)) +
  geom_line() +
  facet_wrap(~ horizon, scales = "free_y") +
  labs(
    title = "Sigma calibration by horizon",
    x = "sigma",
    y = "RMSE within horizon"
  ) +
  theme_bw()
}

```

Grid search over (mu, sigma, h) (v9-style) (optional)

This reproduces v9's idea: jointly tune a global drift `mu`, global volatility scale `sigma`, and smoothing half-width `h`, using a weighted score combining CDC and durable-change mismatch.

```

if (isTRUE(params$run_grid_search)) {

  mu_grid <- c(-0.01, -0.005, 0.0, 0.005)
  sigma_grid2 <- c(0.1, 0.3, 0.5, 0.7, 1.0)
  h_grid <- c(0L, 2L, 5L)

  lambda_cdc <- 0.7

  param_grid <- tidyr::expand_grid(mu = mu_grid, sigma = sigma_grid2, h = h_grid)
  grid_results <- vector("list", nrow(param_grid))

  for (i in seq_len(nrow(param_grid))) {

    mu_i <- param_grid$mu[i]
    sigma_i <- param_grid$sigma[i]
    h_i <- as.integer(param_grid$h[i])

    sm_i <- build_gaussian_params_smoothed(h = h_i)

    sim_i <- simulate_rank_paths(
      w0 = w0_ext,
      K_cut = K_cut,
      K_max = K_max,
      T = sim_T_weeks,
      n_paths = sim_n_paths,
      mu = mu_i,
      sigma = sigma_i,
      entry_frac = sim_entry_frac,
      mean_vec = sm_i$mean_dlogw_s,
      sd_vec = sm_i$sd_dlogw_s,
      horizons = horizons_durable,
      K_xi = NULL
    )

    sim_cdc_i <- sim_i$snapshots %>%
      group_by(rank) %>%

```

```

    summarise(w_bar = mean(share, na.rm = TRUE), .groups = "drop")

rmse_cdc_i <- cdc_rmse(emp_cdc, sim_cdc_i)
rmse_dur_i <- durable_rmse(emp_targets, sim_i$growth, bucket_def)

score_i <- lambda_cdc * rmse_cdc_i + (1 - lambda_cdc) * rmse_dur_i

grid_results[[i]] <- tibble(
  mu = mu_i, sigma = sigma_i, h = h_i,
  rmse_cdc = rmse_cdc_i,
  rmse_durable = rmse_dur_i,
  score = score_i
)
}

grid_tbl <- bind_rows(grid_results) %>% arrange(score)
grid_tbl %>% slice(1:10)

ggplot(grid_tbl, aes(x = score)) +
  geom_histogram(bins = 30) +
  labs(
    title = "Grid-search score distribution",
    x = "score",
    y = "count"
  ) +
  theme_bw()
}

```

Alternative model: nonparametric bootstrap increments (v2/v6) (optional)

This implements the earliest simulator variant: sample each rank-slot increment from the empirical pool of observed dlogw values at that rank (rather than Gaussian).

```

if (isTRUE(params$run_bootstrap_model)) {

  increment_pools <- rank_panel %>%
    filter(rank <= K_max) %>%
    group_by(rank) %>%
    summarise(pool = list(dlogw), .groups = "drop") %>%
    arrange(rank)

  stopifnot(nrow(increment_pools) == K_max)

  sample_increments_bootstrap <- function(mu = 0, sigma = 1) {
    # one sampled increment per rank slot
    inc <- map_dbl(seq_len(K_max), function(r) {
      v <- increment_pools$pool[[r]]
      if (length(v) == 0) return(0)
      sample(v, size = 1, replace = TRUE)
    })
    mu + sigma * inc
  }

  simulate_rank_paths_bootstrap <- function(

```

```

w0, K_cut, K_max, T = 52, n_paths = 200,
mu = 0, sigma = 1, entry_frac = 0.10,
horizons = c(4L, 8L),
K_xi = NULL
) {

top_n_sorted <- function(w, n) w[order(w, decreasing = TRUE)][seq_len(n)]

horizons <- as.integer(horizons)
if (!is.null(K_xi)) {
  K_xi <- as.integer(K_xi)
  stopifnot(K_xi >= 2L, K_xi <= K_cut)
}

snapshots <- vector("list", n_paths)
growth_collect <- setNames(vector("list", length(horizons)), paste0("h", horizons))
xi_collect <- vector("list", n_paths)

for (p in seq_len(n_paths)) {

  if (length(w0) < K_max) {
    w <- c(w0, entrant_sampler(K_max - length(w0)))
    w <- w / sum(w)
  } else {
    w <- w0[1:K_max]
    w <- w / sum(w)
  }

  w_baseK <- top_n_sorted(w, K_cut)
  w_baseK <- w_baseK / sum(w_baseK)

  snaps_p <- vector("list", T)
  xi_p <- vector("list", T)

  for (t in seq_len(T)) {

    dlogw <- sample_increments_bootstrap(mu = mu, sigma = sigma)
    w <- w * exp(dlogw)
    w <- pmax(w, 1e-18)
    w <- w / sum(w)

    w_top <- top_n_sorted(w, K_cut)
    n_tail <- K_max - K_cut

    if (n_tail > 0) {
      n_add <- max(1L, floor(n_tail * entry_frac))
      w <- c(w_top, entrant_sampler(n_add), entrant_sampler(n_tail - n_add))
    } else {
      w <- w_top
    }

    w <- w / sum(w)
  }
}

```

```

wK <- top_n_sorted(w, K_cut)
wK <- wK / sum(wK)

snaps_p[[t]] <- tibble(
  path = p, t = t, rank = seq_len(K_cut), share = wK
)

if (!is.null(K_xi)) {
  w_top_now <- top_n_sorted(w, K_cut) %>% pmax(1e-18)
  xi_vals <- log(w_top_now[seq_len(K_xi - 1L)] / w_top_now[seq(2L, K_xi)])
  xi_p[[t]] <- tibble(path = p, t = t, k = seq_len(K_xi - 1L), xi = xi_vals)
}

if (t %in% horizons) {
  lg <- log(wK / w_baseK)
  lbl <- paste0("h", t)
  growth_collect[[lbl]] <- rbind(
    growth_collect[[lbl]],
    cbind(path = p, rank = seq_len(K_cut), horizon = t, log_growth = lg)
  )
}
}

snapshots[[p]] <- bind_rows(snap_p)
if (!is.null(K_xi)) xi_collect[[p]] <- bind_rows(xi_p)
}

growth_df <- bind_rows(lapply(growth_collect, as_tibble))

out <- list(snapshots = bind_rows(snapshots), growth = growth_df)
if (!is.null(K_xi)) out$xi <- bind_rows(xi_collect)
out
}

sim_boot <- simulate_rank_paths_bootstrap(
  w0 = w0_ext,
  K_cut = K_cut,
  K_max = K_max,
  T = sim_T_weeks,
  n_paths = sim_n_paths,
  mu = sim_mu,
  sigma = sim_sigma,
  entry_frac = sim_entry_frac,
  horizons = horizons_durable,
  K_xi = sim_K_xi
)

sim_boot_cdc <- sim_boot$snapshots %>%
  group_by(rank) %>%
  summarise(w_bar = mean(share, na.rm = TRUE), .groups = "drop")

score_boot <- tibble(
  model = "bootstrap",

```

```

rmse_cdc = cdc_rmse(emp_cdc, sim_boot_cdc),
rmse_durable = durable_rmse(emp_targets, sim_boot$growth, bucket_def),
rmse_xi = xi_rmse(rho_by_k_med %>% filter(k <= sim_K_xi - 1L),
                  sim_boot$xi %>% filter(k <= sim_K_xi - 1L))
)

score_gauss <- score_tbl %>% mutate(model = "gaussian_smoothed")

bind_rows(score_gauss, score_boot) %>%
  select(model, everything())

bind_rows(
  emp_cdc %>% mutate(source = "Empirical"),
  sim_cdc %>% mutate(source = "Gaussian"),
  sim_boot_cdc %>% mutate(source = "Bootstrap")
) %>%
  ggplot(aes(rank, w_bar, linetype = source)) +
  geom_line() +
  scale_x_log10() +
  scale_y_log10(labels = percent_format(accuracy = 0.01)) +
  labs(
    title = "CDC comparison across models",
    x = "Rank (log scale)",
    y = "Mean share within top-K (log scale)",
    linetype = NULL
  ) +
  theme_bw()
}

```

Notes for SPT-facing interpretation (non-quantitative)

- This document computes both **rank-slot** objects (ranked shares and their increments) and **identity-tracking** objects (follow the endpoint currently at rank k forward).
- For SPT / rank-based diffusion discussions, the rank-slot objects are the most direct analog.
- Identity-tracking objects are essential for **turnover** and will often motivate extensions (birth/death, regime switching, common shocks, jumps).